Group 8

Technical Document

Objectives

Primary Objective

The primary objective of this project is to design and implement an electronics test bench. Utilizing a Raspberry Pi 4 as the central control unit the project aims to replicate the core capabilities of traditional standalone benchtop equipment which can be found in the lab. The test bench combines a voltmeter, ohmmeter, square wave, sine wave, frequency counter, and voltage reference all into a single consolidated device.

Key Directives and Deliverables

To achieve this objective, the system design must do the following:

Integrated Control and Real Time Data Acquisition

Develop hardware and software to handle accurate data acquisition and real time device function control

User Interface (UI) Development

Design a concise and intuitive UI that allows the user to easily interact with the device, control its parameters and correctly interpret acquired data

Software

Program the entire system exclusively through Python running the whole system on the provided Raspberry Pi 4 without any additional microcontrollers

Hardware and Power Constraints

Adhere strictly to provided integrated circuit chips. The system must independently derive and regulate all required circuit voltage from a supplied 24V DC main power supply

Constraints

Part Limitation

"Your team may only use components given to you for "active" circuitry* "

*Not including op-amps

This is by far the most limiting factor of the design, since many of these functions can be simply solved by easily buyable ICs online. Therefore, the designs must be focused around intentional circuit design and functionality. Furthermore while the MOSFETs and LM339 comparator are minimal in quantity, the harshest condition is the 2 MCP4131, and 1 MCP4231 for a total of 4 controllable digitally controlled potentiometers provided. Since this design largely requires digitally-controlled variation and ADC functions, ingenuity must be used to save our budget of these ICs for critical components.

Power Limitation

"A 24V AC/DC supply is provided. Your team must derive/design all additional voltage values from this supply only"

The power limitation was an initial struggle. Creating a stable push-pull amplifier was a struggle but not a horrible limitation. In the end after 3 iterations of our push pull amplifier and research we were able to get the stable $\pm$12V supply. However, we then needed to create buffer amplifiers from the stable 12V supply to create a stable 3.3V and 5V source as we could not use the Raspberry Pi source. In the end, this constraint was limiting but not as limiting as the parts constraint after the implementation of the power supplies listed above.

Processing Limitation

"Your team will write code using Python and the Raspberry PI4 provided to you. No additional microcontrollers or microprocessors are allowed."

This was a struggle as our team was inexperienced when it came to the syntax of Python. However, after the first few weeks, the struggle was not as bad. Along with that, the real constraint was that we were not allowed additional microcontrollers. While the raspberry Pi has functionalities similar to other microcontrollers like ESP-32 and Arduino with on board ADC, WiFi, bluetooth and GPIO, the raspberry pi is more powerful with installed IDE and full internet access. While this limitation was difficult

at first, the included IDE's, extra RAM and full internet access were essential for our team's completion of the project.

Hardware

CHECKPOINT C

Digital Potentiometer: Setting a Resistance

Required Parts: MCP4131, MCP4231, RPi 4

Task Breakdown (What)

The MCP 41X1 and 42X1 series chips are digital potentiometers or "digipot" ICs. Like analog potentiometers, terminal A & terminal W have variable resistances from the wiper terminal. However, rather than being controlled by a physical dial, they connect to microcontrollers or microprocessors like the RPi 4 through SPI for digitally controlled variable resistance. While not truly analog, the 10kΩ resistance can be changed to an accuracy of 7 bits, creating small discrete steps to select through.

For this project, the digital potentiometers are used for a wide range of functions, mainly for those that require digitally controlled variable voltage via changes in resistance.

Design Process (How)

We reviewed the MCP41X1/MCP42X1 Digital Potentiometer Datasheet to get a good understanding of how to set up the connections for the ICs as well as the correct protocol to change the resistance across the terminals (see Fig. C1).

We found that while the dual digipot requires a full SPI 4-wire protocol, the pin diagram reveals that the single digipot has a singular DI/DO. Given that we only need to write information to the digipot, we found that we need to connect the RPi 4 MOSI to the DI/DO to write from the RPi 4 into the digipot. Furthermore, by assigning each digipot's CS to a different pin, it allows interfacing between each component separately. When sending commands to the RPi 4, the byte structure reveals that we must send 2 kinds of bytes to cause a change to the digipot: a command byte giving instructions on how to process the following data byte. We used pigpio to handle the spi communication and constructed scripts to automatically set the resistance (see Fig. C2 & Deliverables/Checkpoint_C/Checkpoint_C_Final.py).

Design Methodology (Why)

For the code interface, we used the “pigpio” library because of its ease of use alongside Python code, as well as the environments used on the RPi 4. Isolating how to control the resistance allows us to reuse this function as the project grows in complexity.

Digital Potentiometer: Transfer Functions

Task Breakdown (What)

As stated previously, the digipots are foundational for digitally controlling circuits that require variable voltage or resistance. For this control to be accurate, it is foundational to characterize the true resistance changes across the 128 steps through the 7-bit resolution, as the minimum, maximum, and step resistance vary across each component.

Design Process (How)

To characterize our digipots, first sweep through each of the 128 steps of the MCP4131-103 and MCP4321. To sweep through these resistances, refer back to “MCP41X1: Setting a Resistance”. Now, create a table matching the step to the resistance corresponding to it and plot these points. Now that the date is plotted, find the line of best fit through linear regression (see Fig. C3). We did this by making two arrays with our data points and u

```
% LINEAR FIT
p1 = polyfit(Sb, R, 1);
%Finds the line of best fit
R_fit = polyval(p1, Sb_fit);
%Generates trend line data
R_pred = polyval(p1, Sb);
%Calculates Resistance predicted values
MAE = mean(abs(R - R_pred));
%Calculate Mean Absolute Error
Err = (MAE / signal_span) * 100;
%Calculates linearity error
transfer_equation = sprintf('R = %.2f*bits + %.2f', p1(1), p1(2));
error = sprintf('MAE = %.2f mV', MAE);
```

Using MATLAB’s polyfit function.

(Code works the same for both digipots; what varies are the data arrays used)

Design Methodology (Why)

While this step is incredibly tedious, we thanked ourselves further down the project for characterizing these resistances now, as it alleviated the worry for IC faults when developing the later circuits. Furthermore, it allows you to troubleshoot your individual build between part variations.

KY-040: Characterizing the Rotary Encoder

Task Breakdown (What)

The project requires the user to interface with the device to select different functions from a display. The KY-040 rotary encoder is the user controller to interact with the interface. Therefore, it is important to ensure the device takes reliable inputs in order for ease of use to go through the menu. Furthermore, it is important to implement debounces into these inputs to ensure only intentional inputs control the device.

Design Process (How)

Before plugging the rotary encoder into the RPi 4, we needed to understand the basic functions of the encoder and its expected function from each pin (see Fig. C4).
So for any input, we needed 3 designated GPIOs to the rotary encoder: 2 for rotation, 1 for clicks. Furthermore, it explained how to read the rotations:

Based on whether CLK or DT changed the output first,
The direction of rotation can be determined.

From here, a lookup table was generated for the matching patterns for clockwise (1) or counterclockwise (-1). As the device updates, it looks for these combinations to define these turns. For error handling, it waits for 4 consecutive valid inputs from the turn in order to register these turns in order to register the inputs, eliminating any false inputs. Furthermore, the amount of reads over time would be used to engage between fast and slow scrolls, acting as a quantity change for later menuing.

encoder.py

```
import pigpio

class RotaryEncoder:
   # gray code, same logic as before but interrupt driven
   _TABLE = {
      0b1101: 1,  0b0100: 1,  0b0010: 1,  0b1011: 1,   # CW
      0b1110: -1, 0b0111: -1, 0b0001: -1, 0b1000: -1,  # CCW
   }
 ….
# initialization of pin connections
   def __init__(self, pi, pin_a, pin_b, pin_btn, detent_counts=4):
….
           if delta:
          self._raw += delta
          if abs(self._raw) >= self.detents:
             self._steps += 1 if self._raw > 0 else -1
             self._raw = 0
       self._last_enc = enc
# checks for rotary encoder inputs
```

For the button, the input simply reads either high or low depending on the button presses. In order to debounce the input, a simple 50ms delay for true reads would be used to check whether or not button presses were truly held or ripples from a previous press. This ensured that only when the button was actually held during presses would the input register (see Deliverables/Checkpoint_A/Code/encoder.py and main.py Line 230-253).

Design Methodology (Why)

While not the most optimal conversion, the look-up table is used to save you a lot of time when initially interfacing through the menu. Since the logic is hard-coded into this table alongside a 4 read verification, it allows for very simple usage of reading direction as well as scroll speed readings. This similarly applies to the 50 ms delay for the button press, as it is a minute hard-coded detail that drastically improves menuing without any noticeable delay from the user.

CHECKPOINT B

Push Pull Power Supply

N-MOSFET, P-MOSFET, TL071, Resistors (10kΩ x2, 100Ω, 10MΩ), Capacitors (0.1uF), Diodes (1N4007 x2)

Task Breakdown (What)

The project needs to be completely powered off a 0/24V supply. However, several components of the project require a negative voltage. Therefore, we must create a circuit that transforms the 0/24V to a split +/- 12V rails with a stable virtual ground node for both circuit components and the RPi4 to reference from.

For the project, the power supply needs to have stable +/-12V rails under a max 50mA load, equal or unequal across each rail. Many of the components will be implemented to make sure the voltage is stable.

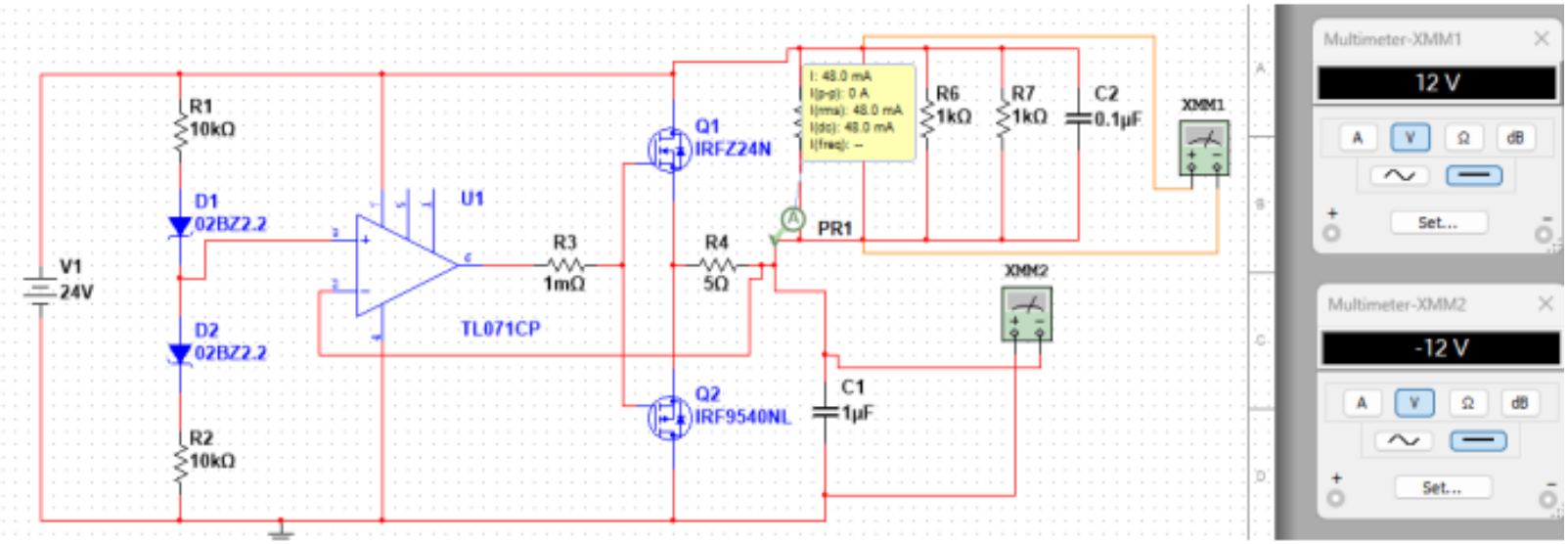
Design Process (How)

A common design to hold this voltage is a class B push-pull amplifier to ensure that spikes from the top or the bottom ends don't have any impact on the power rails (See Fig B1). However, the original push-pull amplifier requires further refinement to properly regulate ±12V rails from a 24V input. Through simulation and debugging, these major clarifications were made:

- Op-Amp Circuit to hold the voltage at a the midpoint, 12V or virtual ground
- Addition of a large resistor (1-10MΩ) for stability
- Addition of a small resistor (5-100Ω) for preventing short circuits
- Decoupling Capacitors across the load for load stabilization

All of these are implemented into the current schematic with a voltage divider to split the voltage in half and diodes to direct the current flow, an op-amp circuit to hold split voltage (12V), a NMOS and PMOS to push and pull voltage, and capacitors to eliminate voltage spikes on the load. Ensure the MOSFETS have their gate pins connected at the end of the Op-Amp large resistor, source pins connected at the end of the small resistor, and the drain pins are connected to their respective voltage rail.

Note: Ensure this circuit is electrically isolated from the rest of the components during testing over a breadboard or when designing over a PCB, so that when if the push pull amplifier does not work or push incorrectly, your components are safe



On simulations, the power rails maintained a +/- 12V stable output under 1kΩ load within simulation, loaded or unloaded on both sides. For breadboard testing, a power supply was used to set the load out to 50mA, from which the rails performed ~11.8V at 47mA when loaded and ~12.2V at ~0mA unloaded, well within supply range for the components (See Fig. B2).

Design Methodology (Why)

- Voltage Divider with Diodes

Gives the operational amplifier a clear midpoint to keep stable while ensuring proper current flow.

- Class B Push-Pull Amplifier

This is a common setup to deal with variations in input voltage for a voltage rail splitter. As the load voltage shifts away from the split voltage, the N-MOSFET pushes additional current from the +12V supply rail down to the load, aka virtual ground. Inversely, when the divided voltage spikes, the P-MOSFET pulls the extra current away from the virtual ground into the -12V supply rail.

- Large Biasing Resistor

The power supply is prone to several changing oscillations that can cause the MOSFETS to work harder. A large 1-10MΩ resistor lags these oscillations seen by the MOSFETS down to reduce the high-frequency switching.

- Small Isolating Resistor

It is important that the push-pull amplifier is stable under changing loads to keep the components safe. This resistor does exactly that by preventing potential shorts while also again further stabilizing the loads.

- Decoupling Capacitors

Again, the power supply is prone to spikes and oscillations, causing noise. The addition of decoupling capacitors on both rails eliminates this high-frequency noise and spikes from load switching, allowing for a more stable rail.

Ohmmeter: Wheatstone Bridge & Digipot

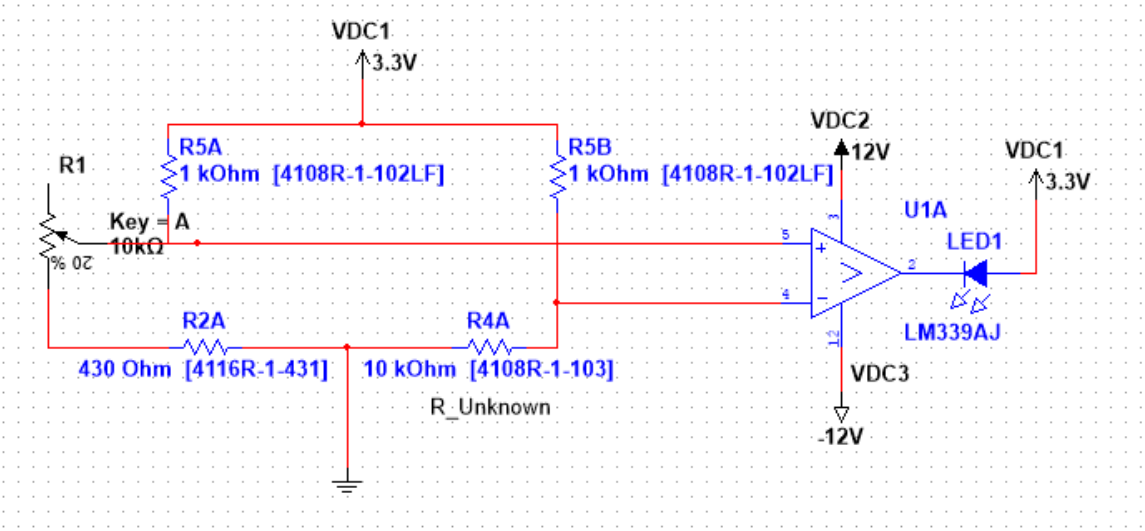
MCP4131, Resistors (10kΩ x2, 430Ω), LM339

Task Breakdown (What)

The power bench must include an ohmmeter that autoranges inputs across a 500Ω-10kΩ range within a 50 ±YΩ range. The ±YΩ range is revised from your unique digital potentiometers' resistance steps characterized previously. The LCD must display the measured value to the user alongside a derived accuracy tolerance based on ±YΩ.

Design Process (How)

First and foremost, we referred to our prior digipot characterization to find the ohm steps and the intercept resistance. Using the characterized values, we used a Wheatstone bridge and LM339 comparator to create 2 common voltage dividers between the inserted unknown resistor and the digital potentiometer. For the digital potentiometer voltage divider, calculate the low-end series resistor by subtracting 500Ω from the intercept's resistance and matching it to a nearby real resistor. For our components, 500 - 104 = 396, where we substituted an available 430Ω resistor.



On the RPi 4, the digital potentiometer is scrolled through from step 0 until the resistance subsequent;y matches the unknown resistor, flipping the comparator and sending a high signal to the RPi 4. From here, a measurement algorithm tracks which step caused the comparator to trip off, and the corresponding resistance is returned using the step_to_res() transfer function. An abort function (the encoder button) is passed to the sweep loop so the user can cancel a long measurement without waiting. res_to_step() is the inverse: given a desired resistance, it solves (R − intercept) / slope and clamps to 0–127. Both functions are used to set known resistances during characterization and to read back measured values (see Fig. B3)

Design Methodology (Why):

- Wheatstone Bridge

A common design is creating a voltage divider directly with the unknown resistor and the digital potentiometer, subsequently scrolling from the lowest bit up to an eventual split voltage of 1.65V from equal resistance. From here, the same comparator architecture and transfer function would solve for the resistance.

However, this design is prone to noise since all the components are in line. By using the Wheatstone bridge, we can still integrate the simple application and algorithm of the simple voltage divider and if there is noise in the lines, it will go to both and effectively cancel out the noise!

- Resistor Sizing

The top side of the voltage divider for both Wheatstone bridges was selected as 1kΩ since they provided a high level of accuracy when reading smaller resistance values from 500-900Ω while maintaining good resolutions at higher resistances from 9-10kΩ.

For further support on the lower resistances, the 500 - intercept resistor isolated the changes in the digital potentiometer to directly correlate to the steps

above the lowest read value. This eliminates further inaccuracies from direct resistor step translation off of the digital potentiometer, only focusing on the steps above the known minimum.

- Code Logic

The ohmmeter auto-ranging is a very simple search with the digital potentiometer. The code directs the digipot to scroll from step 0 i.e. its lowest resistance, all the way up until a comparator reading flips high. Using the characterized transfer function to find the step's equivalent resistance, the digipot's resistance is added to the low series resistor to calculate & display the calculated resistance. When exiting the UI menu for this section, the digipot is reset to step zero to avoid residual bridge imbalance affecting other measurements.

The SPI communication with the ohmmeter is very fast at 100kHz, so while a more sophisticated search algorithm can be used to speed up the resistance readings, we instead opted for a simple algorithm due to robustness. In turn, the correct resistance updates every 2 seconds on the LCD.

Voltmeter — 6-Bit SAR ADC

Resistors (10kΩ x18, 5kΩ x5, 2kΩ x2, 1kΩ), TL071 x3, LM339

Task Breakdown (What)

The power bench includes a voltmeter that measures DC voltage from +/- 5V at a +/- 0.3125V accuracy from either an external voltage source or from the internal DC voltage reference and displayed on the LCD UI. It is important to note that both of these readings need to source from the same logical ADC unit.

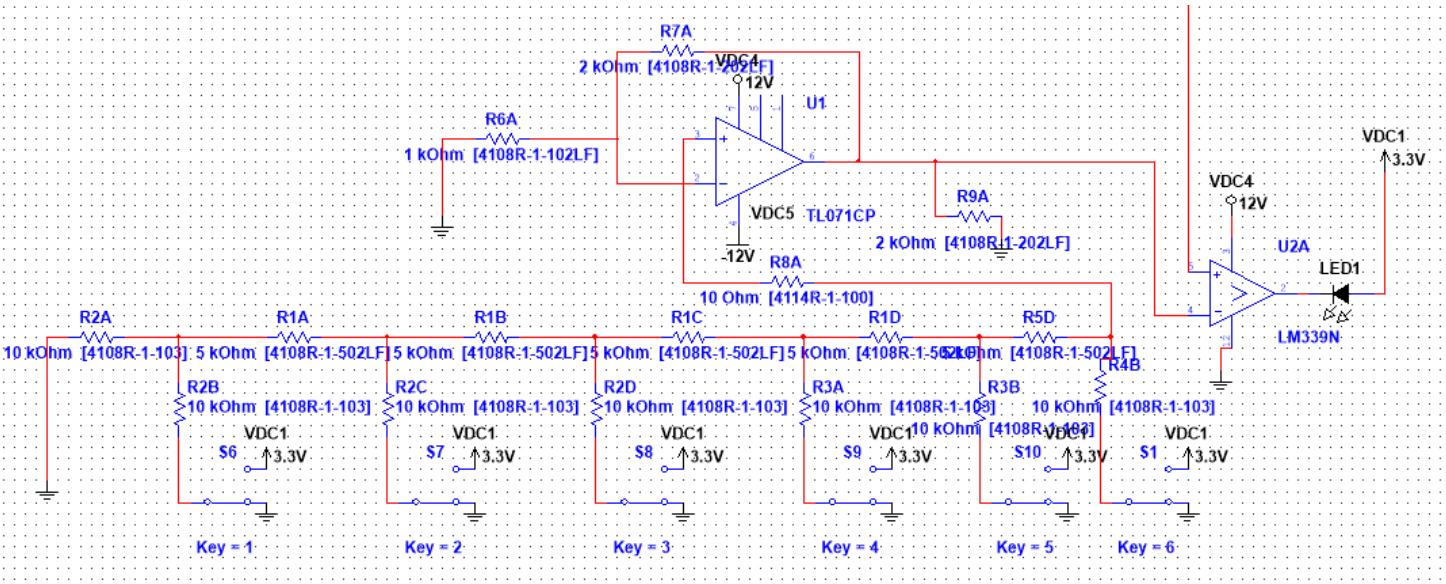
Design Process (How)

Due to the digipot's limited quantity and inability to see negative voltage, this component of the voltmeter was made with this restriction in mind while maintaining the necessary resolution. Note, this across +/-5V with +/- 0.3125V accuracy is 6-bit resolution.

With a limited GPIO count, we used an R-2R ladder to create a SAR ADC with the RPi 4 to run a binary search across 6 bits. The ladder divides a common input voltage across its 2R input resistor. While any R & 2R value technically works for the ladder, we suggest using 5kΩ and 10kΩ resistors for a clean response. The R-2R ladder is a combination of several voltage dividers scaling down the input voltage by a power of 2 with each subsequent rung. As the 3.3V voltage turns on across each pin, it scales

from 3.3 * 1/(2^1) = 1.65V at the MSB all the way down to 3.3 * 1/(2^6) = 0.051V at the LSB for a total of 6 bits. By combining the voltages by turning on any assortment of pins, you can reach any voltage up to the common input voltage, up to the resolution of as many rungs as there are on the ladder. By creating a 6-run ladder, it implements a 6-bit resolution needed for the voltmeter to work.

The RPi 4's 3V3 output 6-bit resolution would be amplified up to 10V through a non-inverting amplifier op-amp circuit. This amplified 6-bit response from 0/10V would be sent into separate comparators to read either the external or internal reference.



In order for an appropriate comparison, the read voltage would be amplified by 5V scaling +/- 5V reference to the common 0/10V into the comparator through a summing amplifier circuit (see Fig. B4). With the LM339 output and 2R connected to the GPIO, the RPi 4 would find the voltage, reading the comparator output to estimate the voltage up to the same 6- bit accuracy. For the calculations for the display, the voltage reading readjusts the 5V offset, converting 0/10V back down to +/- 5V (see Deliverables/Checkpoint_A/Code/meter_hardware.py).

To find the voltage, the SAR ADC performs a 6-bit binary search. Starting with the MSB bit 5, it tries each bit in decreasing order. In each trial, the DAC code is written, and the comparator output is read after a settling delay. If the comparator indicates the DAC voltage exceeds the input, the bit is kept; otherwise, it is cleared. After six iterations, the 6-bit code is converted to voltage using the characterized linear equation (see Deliverables/Checkpoint_A/Code/meter_hardware.py).

The voltmeter has two source modes selectable from the UI. In External mode, the comparator input is connected to the external measurement terminal. In Internal Reference mode, the DC reference output is similarly connected to a comparator

input, allowing the system to verify its own DC output accuracy. Testing consistently showed expected values to measure within +/-5V at approximately 5% accuracy, consistent with the 6-bit resolution (See Fig. B5 & B6).

Design Methodology (Why)

- ADC

While a flash ADC would be optimal for speed, neither the GPIO budget nor the comparator count supports the voltmeter ADC at this resolution. Therefore, the R-2R SAR ADC accommodates both the speed necessary to read voltage live as well as the relatively low GPIO budget of only 8 pins (6-bit R-2R + 2 LM339 for internal & external voltage). Also, it helps to save on the restricted digipot budget, which is needed for output components like the internal voltage reference and the function generator's square and sine wave.

Measured Voltage

The R-2R ladder only works across positive voltages, so while the ladder can be scaled up to a similar amplitude, the measured voltage must be brought above zero to be accurately compared against the R-2R DAC for the ADC to work properly.

Since we found the full range of the measured voltage to be 10V, we amplified the R-2R signal up to 10V, matching the 0.3125V step size. Furthermore, each source was given a summing amplifier into its own comparator terminals, adding the new midpoint voltage of 5V to properly scale the voltages from 0/10V to match the R-2R ladder.

- Code

Connecting GPIO outputs to each leg of the R-2R ladder, the RPi 4 runs a binary search to segment the analog voltage into discrete steps. It does this by going from MSB to LSB, looking at the comparator output to determine whether or not it flips high to decide whether or not to keep each subsequent bit. After 6 rounds, the code finds the voltage accuracy within 6-bit resolution. Furthermore, a final calculation for the display voltage accommodates the 5V midpoint offset, converting 0/10V back down to +/- 5V.

DC Voltage Reference

MCP4131, TL071, Resistors (10kΩ x3)

Task Breakdown (What)

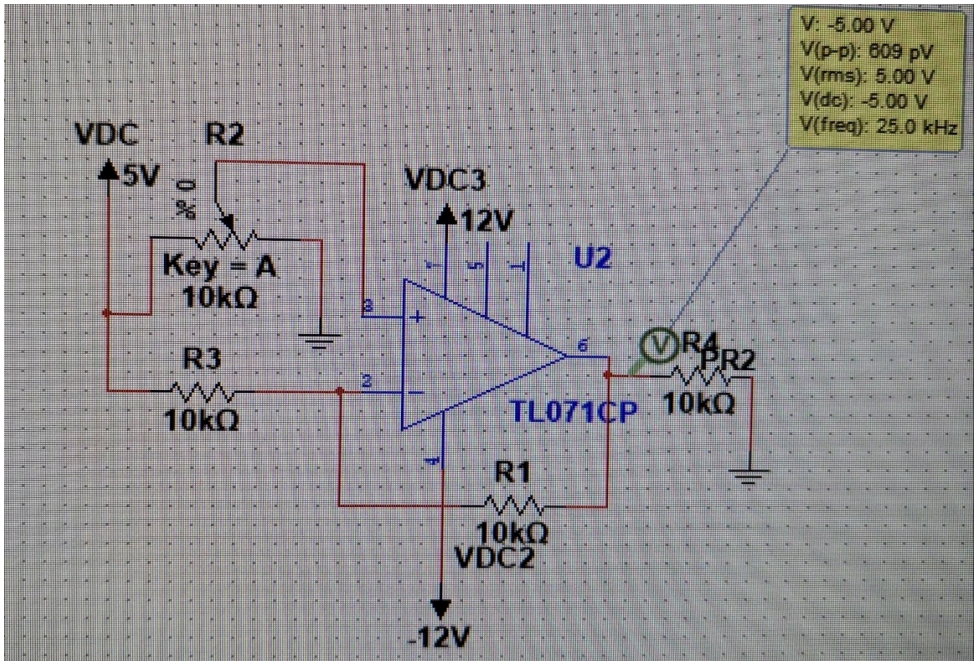
The testbench must be able to generate a DC voltage reference from -/+ 5V with a resolution of 0.625V steps, controllable through the UI by the user input via the

rotary encoder. This reading must be accessible from the DC reference screen, internally from the voltmeter, and externally for additional measurement, and the output must automatically turn off when backing out into different menuing. Note that this reference is not expected to supply high levels of current and assumes a high Z external measurement.

Design Process (How)

To reach digitally controlled variability, the MCP4131 digipot is used as a voltage divider, where the wiper voltage is divided from a 5V source voltage (prior derived from the 12V rail) across the 7-bit resolution of the device. This is well within resolution for the 0.625V steps and will serve for heightened accuracy of the reference.

With the digitally-controlled potentiometer varying 0-5V input, the difference amplifier will output the difference between the positive terminal times 2 vs the negative terminal.



The code translates the desired voltage into digipot steps. This is done by biasing up the desired voltage from -/+5V to 0/10v, aligning with the non-inverting amplifier's output without the 5V source offset, since it is easier to work with. The desired voltage would be found as a percentage of the full 10V, and the same percentage would be taken out of the full digipot steps, accurately converting to the input voltage within the 0.625V steps due to the high resolution of the digipot.

Furthermore, the DC output screen shows a live voltmeter measurement when the output is on (see Fig. B8). This is implemented in the main loop as a separate measurement branch that calls measure_dc_output() every 0.5 s when the output is active and the user is idle. The result is stored in state['dc_measured'] and shown by _draw_dc_output() alongside the setpoint.

meter_hardware.py

```
def set_dc(pi, sd, voltage):
    voltage = max(-5.0, min(5.0, voltage))
    step = round(((voltage + 5.0) / 10.0) * 127)
    step = max(0, min(127, step))
    print(f"set_dc: voltage={voltage:+.3f}V  digipot step={step}")
    _write_pot(pi, sd, step)

def clear_dc(pi, sd):
    _write_pot(pi, sd, 64)

def measure_dc_output(pi, sd, setpoint_voltage):
    set_dc(pi, sd, setpoint_voltage)
    time.sleep(0.1)
    measured_v, code = measure_volts(pi, comp_pin=COMP_PIN_INT)
    return measured_v, code
```

Design Methodology (Why)

A different amplifier was chosen because of it being cheap in terms of components, it would keep the digital potentiometer within its safe operation range of 0V to 5.5V, and it is a concept we had learned about in previous classes. It also solved our original problem of how to determine if the user wanted negative voltage vs positive voltage without using a transistor that would have a ≈0.7V drop.

$\frac{5-Vdgi}{R3} + \frac{Vout-Vdigi}{R1} = 0$

$R1 = R3 \therefore$

$(5 - Vdigi) + (Vout - Vdigi) = 0$

$Vout = 2Vdigi - 5V$

Through the KCL above, we can see that the simplified formula for this difference amplifier shows how 5V to -5V can be achieved without damaging the digital

potentiometer. By choosing $Vdigi = 0$, $Vout = -5$. Now, if instead you use the max voltage, 5V, the output will be 5V!

Square Wave

MCP4231, TL081CP, Capacitors (2.2µF), Resistors (56kΩ, 10kΩ, 100kΩ)

Task Breakdown (What)

Generate a square wave with variable amplitude of +/- 10 Volts centered on the x-axis with a 50% duty cycle. The frequency should be adjustable from 100Hz-10KHz in 10Hz resolution. Use the rotary encoder and LCD UI to control the amplitude and the frequency. The resolution for peak values and center frequency (x-axis) should target within 0.3125 volts.

Design Process (How)

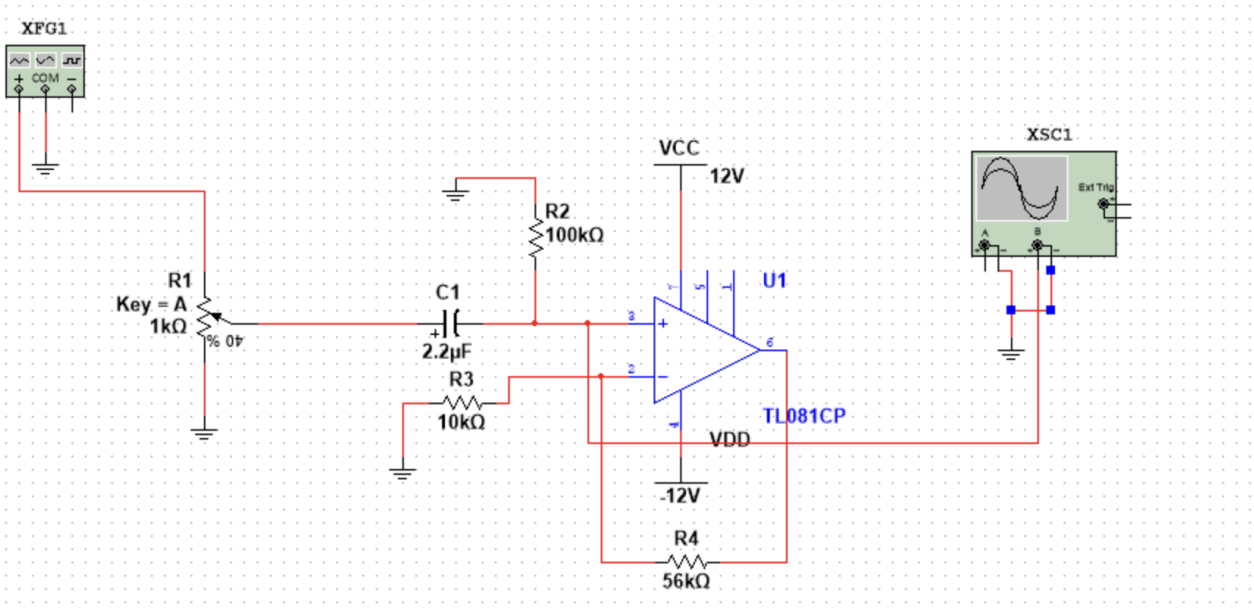
In order to design a square wave centered on the x-axis we had to design a way to turn 0-3.3V 50% duty cycle pwm signal into a signal centered on the x-axis with a variable amplitude. Giving this pwm signal a variable amplitude required the use of our dual digipot (MCP4231),here the signal was routed directly to one of its terminals while the other was grounded, creating a voltage divider. From the wiper the decreased amplitude wave is outputted.

The wave output is still positive so we had to design a way to give it the right DC offset at each amplitude to center it on the x-axis. To achieve this we passed the wipers output through an AC coupling circuit, a capacitor in series with a resistor tied to ground. Since a capacitor blocks the DC component of the signal the variable positive offset is stripped away while the grounding resistor sets a 0V baseline. This automatically centers the AC signal of the square wave to the x-axis regardless of the amplitude set by the digipot.

Now that the signal is centered on the x-axis amplification is needed to reach the +/-10V requirement. To do so a non-inverting op-amp (TL081CP), powered with +/-12V, with a gain of 6.6 was used (1+R2/R1 →1+56kΩ/10kΩ) which takes the maximum signal of +/- 1.65V to 10.89V, giving us some headroom for our maximum values.

To reliably generate the PWM signal without any software timing jitter the square wave's frequency is controlled through the Raspberry Pi's PWM controller using the pigpio library on GPIO 18. When a frequency change is requested through the UI the software first ensures its within the specified operating range of 100Hz-10Khz. This input is then passed to the hardware_PWM method which has a constant duty

cycle of 50%. In testing, we found accuracy within 5% for both the amplitude and frequency of the signal (See Fig. B7)



fg_hardware.py

```
import pigpio

# Pin and PWM constants
PWM_PIN = 18           # Square wave output (hardware PWM capable)
PWM_DUTY_50 = 500_000   # pigpio uses 0-1_000_000 range, 500_000 = 50%

def _pwm_set(pi, freq):
    """Drives the square wave via pigpio hardware PWM."""
    global _pwm_freq, _pwm_on

    if freq == _pwm_freq and _pwm_on:
        return

    if freq > 0:
        # pi.hardware_PWM(gpio, frequency_hz, duty_cycle_0_to_1000000)
        pi.hardware_PWM(PWM_PIN, freq, PWM_DUTY_50)
        _pwm_freq = freq
        _pwm_on = True
    else:
        pi.hardware_PWM(PWM_PIN, 0, 0)
        pi.write(PWM_PIN, 0)
        _pwm_freq = 0
        _pwm_on = False
```

This snippet shows the interface with the Pi's hardware timers and the locked 50% duty cycle.

fg_hardware.py

```
def fg_start(pi, sd_fg, frequency, amplitude, waveform="square"):
    #sine wave handling omitted for brevity
    else:
        # stop sine before starting square
        sine_stop()

        # Clamp input frequency between 100 Hz and 10 kHz for hardware stability
        freq = int(max(100, min(10_000, frequency)))

        # Apply the clamped frequency to the hardware PWM
        _pwm_set(pi, freq)

        #amplitude digipot logic omitted for brevity
```

This snippet (an excerpt from your fg_start function) shows how the user input is safely bound before being passed to the hardware.

Design Methodology (Why)

- Hardware PWM over Software Timing

By using pigpio to interface with the Pi's hardware PWM timer on GPIO 18 we guaranteed that our output was jitterfree at the specific frequencies we wanted it at. Additionally routing both of the square wave and sine wave through pigpio eliminated a pin-ownership conflict with the kernel sysfs PWM which had caused our sine wave output to fail after the square wave was generated.

- Passive Scaling and Zero-Centering

Rather than scaling the amplitude and then subtracting a moving DC offset we decided to use a passive to active pipeline. The MCP4231 digipot scales the 0-3.3V wave by using a simple voltage divider, this ensures that the digipot doesn't see negative voltage, something it's not rated for. Passing this positive-only signal through an RC AC-coupling circuit stripped away the variable DC offset centering our wave on the x-axis. This wave was then amplified with a non-inverting op-amp to achieve a +/-10V square wave.

CHECKPOINT A-

Sine Wave

MCP4231, 2 Op-Amps (TL081CP), Capacitors (1nF, 2.2nF, 0.01µF), Resistors (2 6.8kΩ, 2 1kΩ, 2.7kΩ)

Task Breakdown (What)

Generate a sine wave which outputs positive voltage only, 0-10V, variable amplitude in 0.625V steps. A frequency that is variable from 1K-10KHz, variable by 500Hz.

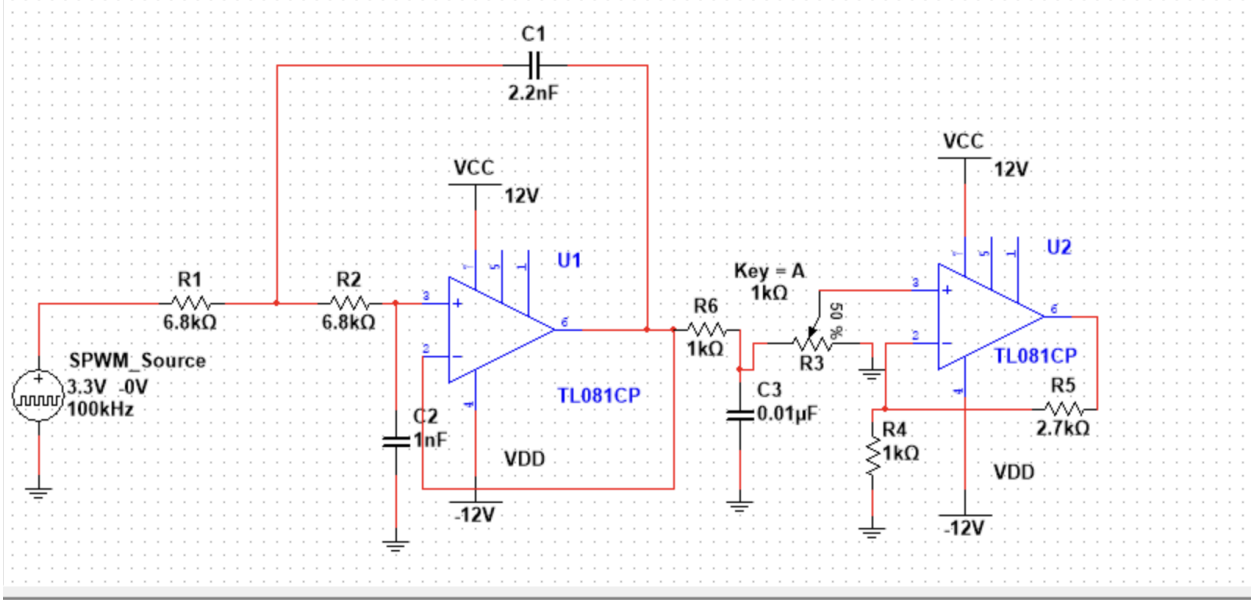
Design Process (How)

Unlike a square wave which can be generated with a simple on/off signal and a constant duty cycle, generating a sine wave digitally requires Sinusoidal Pulse Width Modulation (SPWM). To achieve this we generated, through code, a high-frequency carrier wave whose duty cycle varied constantly to plot out the analog curve of a sine wave

Since this raw SPWM signal is a high speed harsh 0-3.3V digital pulse it must be smoothed out. To do this we routed our SPWM signal from GPIO 13 into a 2nd order Sallen-Key active low-pass filter. Using the first TL081CP op-amp as a unity-gain buffer alongside two 6.8kΩ resistors and a 1nF capacitor to ground and a 2.2nF feedback capacitor the filter created a steep -40dB roll-off or a cutoff frequency of around 15.8kHz which was calculated by $f_c = 1/(2\pi*\sqrt{R1R2C1C2})$. This filtered out the carrier fuzz and left behind a smooth 0-3.3V sine wave. Although the filter did its job the wave was still noisier than expected so as a last measure we passed the signal through an RC low-pass filter (a resistor in series with a grounding 0.01µF capacitor)

To change the amplitude of this sine wave similar to the square wave we used the second half of our digipot (MCP4231) as a voltage divider with one terminal receiving the sine wave, one grounded, and the wiper being the scaled sine wave.

Finally to reach the 0-10V requirement the sine wave was passed through a second op-amp configured as a non-inverting amplifier with a gain of 3.7 which scaled our maximum of 3.3V to 12.21V, this was done because at higher frequencies the wave generated from our Sallen-key filter would have a decreased amplitude.



In order to generate our SPWM output through code we pre-calculate a table, which would be an array of 100 data points representing a perfect sine wave. In this table each point contains the exact ON/OFF duty cycle percentage. To calculate the angle we would use the formula Angle = (step/samples)*2π. For every step the script calculates the exact angle across the sine wave. Then the script calculates a value from -1 to 1 given by x =sin(Angle). This outputs a “negative” duty cycle so to fix this we push up the entire wave by adding 1, giving us an output of 0-2, and scale it to 0-1 by dividing it by 2. Giving us the percentage for the duty cycle. The way we can change the frequency using this duty cycle calculation is by changing how fast the Pi reads the 100-step array. If it reads it 1000 times per second we output a 1kHz sine wave If it reads it 10000 times per second we output a 10kHz sine wave (see Deliverables/Checkpoint_A/Code/spwm.py). Since the main code loops run on top of the S/PWM function, we isolated the the code, ensuring the hardware PWM block is safely disabled before the DMA engine takes over the internal clocks for the sine wave (see Deliverables/Checkpoint_A/Code/fg_hardware.py).In testing, we found the frequency was accurate within 1% of the target frequency, and the amplitude was accurate within 5% of the target amplitude (see Fig. A1).

Design Methodology (Why)

- Active 2nd-Order Filtration & Passive Smoothing

We chose an active 2nd-order Sallen-Key rather than a simpler 1st-order passive RC filter. A 1st-order filter only provides a -20dB/decade roll-off, which allows too much of the high-frequency digital carrier "fuzz" to bleed into the sine wave. By introducing the first TL081CP op-amp and the 6.8kΩ resistor and 1nF and 2.2nFcapacitor pairs, we

achieved a strict -40dB/decade cliff. We placed this before the MCP4231 digipot to ensure our no voltage ripples and negative passed through it.

Frequency Measurement

LM339, Resistors (1kΩ, 100Ω)

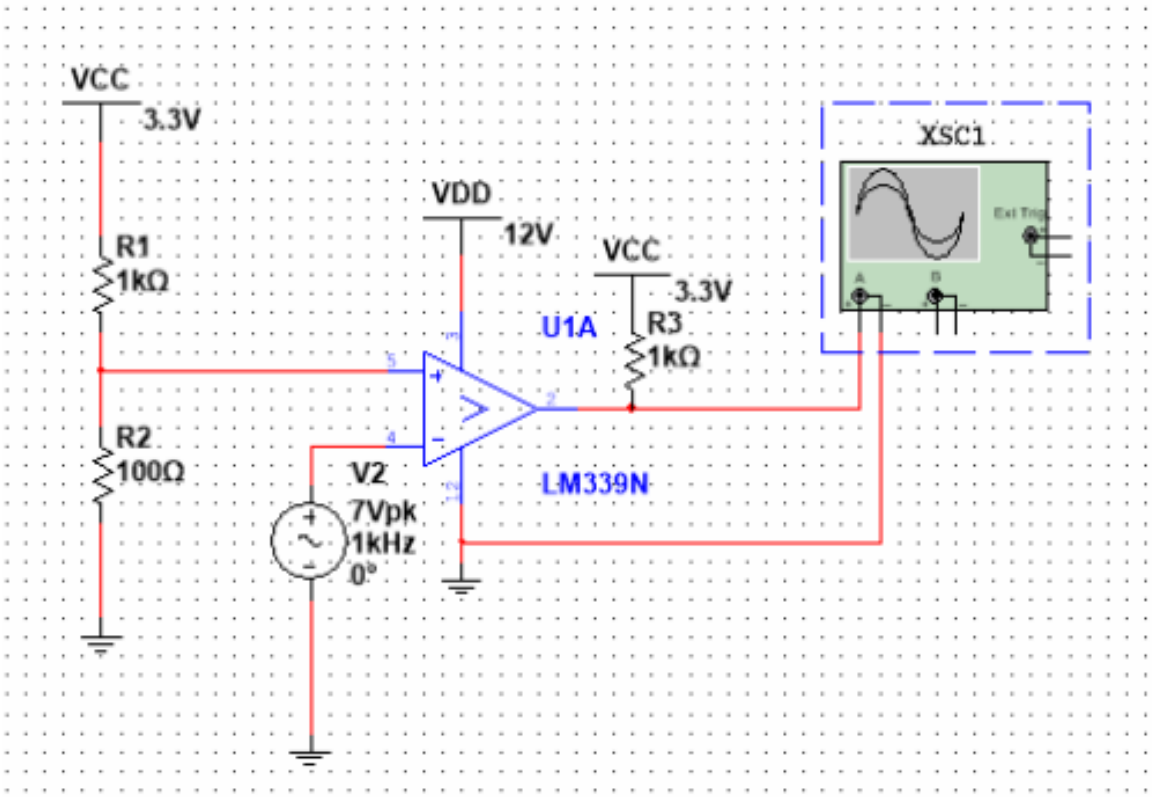
Task Breakdown (What)

Expanding on the AC functions of the testbench, the device must be able to receive a positive oscillating signal and measure its frequency from a range of 1-10kHz.

Design Process

Fortunately, the RPi 4 GPIO pins can read from 1-10MHz, well within the necessary read frequency of the RPi4. All we needed to do was design a circuit that creates highs and lows for the RPi4 to read over time to calculate the frequency.

Since LM339's max operational frequency is around 75-100kHz, we created a simple LM339 comparator circuit, comparing a small voltage (~0.3v) divided from the 3V3 rail against the input signal. This results in the comparator firing from low to high only between periods. The code looks for this rising edge to transform elapsed time into frequency to be later displayed on the LCD (see Deliverables/Checkpoint_A/Code/meter_hardware.py). During trials, we found the readings to be near 98% accurate from 500Hz - 12kHz (see Fig. A2).



Design Methodology

- Time-based reading

Designs based on read frequency are purely dependent on the hardware, and since both the LM339 and RPi4 GPIOs exceed the frequency cap of the necessary read frequencies, we decided to implement this solution due to ease of implementation for both hardware and software.

SOFTWARE

Checkpoint C

Task Breakdown (What)

For this checkpoint, the testbench uses a RPi 4 to control 2 digital potentiometer's resistance through user input via a KY-040 rotary encoder, displaying the value on the LCD display. The UI must support a menu to control each digipot's resistance independently of each other. The user inputs into the rotary encoder must follow "short" presses (<3 seconds) to select items in the menu & "long" presses to return to the primary menu, and it must change resistance by 10Ω with "slow" turns and by 100Ω by "fast" turns. The code must be solely interfaced by the rotary encoder.

Logic Overview (How)

In order for the RPi 4 to interface with external components through its GPIO pins, we use the library "pigpio" to initialize the states of these pins as input for external inputs like the rotary encoder turns and clicks (see Deliverables/Checkpoint_C/ encoder.py), SPI to use the dedicated SPI pins to communicate with the digital potentiometers, and I2C to use the dedicated I2C pin to communicate with the LCD display. Furthermore, the LCD display uses the library "rplcd" to transform strings into the display on the component (see Deliverables/Checkpoint_C/lcd_display.py).

For the device to operate independently, the code must initially set up a launcher file with a "cron" tab. The launcher file was written as a bash file that initializes hardware, ensures SPI readiness, restarts pigpio, and then runs the main program in its current directory. It was developed incrementally by adding delays, hardware checks, and logging to fix startup issues and errors for debugging.

crontab -e → this to open the crontab editor @reboot /home/group8/launcher.sh -> add this following line to and save the edit Chmod +x /home/group8/launcher.shd -> verifies executable

Functionally, the launcher script runs automatically on bootup using crontab. It will wait for the RPi to fully start up and load the OS, then check that SPI is ready and restart the pigpio daemon and ensure hardware commutation works. It moves into the project folder, runs a short text execution to initialize the hardware, and then starts the main program. The program runs continuously in the background while logging activity from the user in case there are any errors that occur (see launcher.sh)

As the program runs, the double digipot (MCP4231) is set to its starting state, from which the user can select through the menu with the rotary encoder by rotating the dial to cycle through options and clicking to select options (see Deliverables/Checkpoint_C/lcd_display.py).

Once a digipot is selected, the same rotary encoder is used to change resistance, determined by the direction of the rotation (increase/decrease) and the speed of the rotation for (small 10Ω steps vs large 100Ω steps) (see Deliverables/Checkpoint_C/Checkpoint_C_Final.py Line 141-145).

As the UI is selected through to set resistance, the LCD continuously updates to display the target resistance value, and the digital potentiometer updates its resistances in steps after each change in resistance (see Deliverables/Checkpoint_C/lcd_display.py Line 33-37).

A transfer function was used to transform the set resistance values as a percentage of maximum resistance, where the percentage would then be applied to the maximum steps of the potentiometer to get a close approximation to the desired resistance (see Deliverables/Checkpoint_C/Checkpoint_C_Final.py),

Checkpoint B

Logic Overview (How)

The software for Checkpoint followed a more modular design, using a single main file that called multiple helper files in iteration. Each helper file focuses on one specific task to complete within the checkpoint and is created to be combined with other tasks in the main.py file. The rotary encoder and lcd are required to work together to gather user input and manipulate the output being shown accordingly. The components added included the Ohmmeter, Voltmeter, and DC Reference. The pigpio library is used for gpio and spi communication while rplcd is used for controlling the 20x4 display. The time library is used for timing and syncing throughout the files.

One of the main files implemented in the running file is “menu_definitions.py”. This file defines all possible menu situations and generates the UI tree as the user provides inputs. The menu state is tracked, allowing the hardware to be activated as needed to avoid causing more work than needed.

See Deliverables/Checkpoint_B/FinalCode/menu_definitions.py

Default States

```
DEFAULT_STATE = {
    "current_menu":   "mode_select",
    "menu_selection": 0,
    "needs_update":   True,

    # input mode - used when user is typing a number
    "input_mode":       False,
    "input_type":       "",
    "input_value":      "",
    "input_target":     "",
    "input_typed":      "",
    "input_palette_idx": 0,

    # function generator
    "fg_type":      "Square",
    "fg_frequency": 1000.0,   # hz
    "fg_amplitude": 3.3,      # vpp
    "fg_output":    False,

    # ohmmeter
```

```
    "ohm_reading":   0.0,
    "ohm_threshold": 1000.0,  # ohms

    # voltmeter
    "vm_source":      "External",
    "vm_reading":     0.0,
    "vm_threshold":   5.0,    # volts
    "vm_has_reading": False,

    # dc reference
    "dc_voltage": 3.3,
    "dc_output":  False,

    # set True when a setting changes so hardware sync knows to push it
    "hw_update": False,
}
```

The rotary encoder is used to take the user input and navigate the menu as the state and display is updated. These inputs are handled by the “navigation.py” file. The lcd is updated throughout the UI tree based on the user input. The rotary encoder scrolling is interpreted and applied to the menu definitions to update the lcd as needed. Each selection has its own tree that is then pushed to the front until the return buttons are pressed.

Rotary Encoder Scroll

```
def scroll(self, d):
        menu  = self.st["current_menu"]
        items = MENUS.get(menu, [])
        if not items:
            return
        sel = (self.st["menu_selection"] + d) % len(items)
        self.st["menu_selection"] = sel
        self.st["needs_update"]   = True
```

Selection Process

```
        elif picked == "DC Reference":        s["current_menu"] = "dc_reference";
s["menu_selection"] = 0
        elif picked == "Voltage Value Input": self._start_input("Voltage", "dc_voltage")
```

- This is just a snippet but it’s the overall idea of menu navigation based on user selection

The User Interface is controlled and set up by the “navigation.py” file but is ultimately displayed to the LCD by the “ui_renderer.py” file. This file is designed to control what the user should be shown depending on inputs, current states, and menu selection. Renderer works with “lcd_display.py” to update the software of the lcd UI and then alter the physical display to match. The render file updates based on the stage being presented and follows the given UI tree to mimic the changes shown in navigation based on user input. The display file connects to the lcd screen using I2C and is formatted internally so that the displayed lines are not cut off and oriented smoothly within the screen.

See Deliverables/Checkpoint_B/FinalCode/ui_renderer.py
See Deliverables/Checkpoint_B/FinalCode/lcd_display.py

The largest part of this functional decomposition is the files that handle individual user inputs for each hardware function. The files “input_handler.py”, “fg_hardware.py”, and “meter_hardware.py” all work together to create a cohesive hardware output based on software. The input file is what actually deals with user scrolling on the rotary encoder while hardware is active. The different hardwares allows for different alterations to be made to voltage and frequency of the output. This file works to understand what the user wants to change and how based on the encoder input and the menu screen active. The Function Generator Hardware file is what handles the function generator output. This file uses the SPWM pin and pigpio to produce a square wave by turning the pin on and off periodically. The most extensive file of these is the meter hardware file. The voltmeter, function generator, ohmmeter, and dc reference are all compiled into this file to make the implementation simpler. Each file is added as it is created before being added to the main file. This file handles all frequency readings and measurements as well as amplitude for respective hardware. The voltmeter and ohmmeter are also constructed here, using the digipot and SPI connection to accurately take readings to be displayed in other files.

See Deliverables/Checkpoint_B/FinalCode/input_handler.py
See Deliverables/Checkpoint_B/FinalCode/fg_hardware.py
See Deliverables/Checkpoint_B/FinalCode/meter_hardware.py

- Ohmmeter: lines 48 - 131
- Voltmeter: lines 168 - 190
- DC Reference: lines 191 - 201

The software is operated in a continuous loop in the main program that constantly reads user input and updates the system state, syncinging the hardware and display . When the system starts, it initializes all hardware components including GPIO pins and spi connections, the Digital analog converter and the lcd display and loads a default system state. The user interacts or inputs with the system using the rotary encoder and where rotation is detected by the encoder.py file and proceeds with the navigation to move through menus or adjust values. This system also includes safety features such as auto turn off outputs when navigating back or returning the main menu. The continuous loop keeps user input and display outputs and hardware behavior fully synced.

See Deliverables/Checkpoint_B/FinalCode/main.py

Safety Features

```
if held > 0.05:
                if held >= MAIN_HOLD:
                    if state.get("value_scroll_mode"): # short press confirm, longer hold
cancel
                        cancel_scroll_value(state)
                    nav.go_main()
```

- Following snippet is from Navigation.py

```
def go_main(self): # go to main menu and turn everything off
        self.st["fg_output"]       = False
        self.st["dc_output"]       = False
        self.st["vm_source"]       = "External"
        self.st["hw_update"]       = True
        self.st["current_menu"]    = "main"
        self.st["menu_selection"]  = 0
        self.st["needs_update"]    = True
        self.st["ohm_park_needed"] = True
        print("went to main - outputs off")
```

Checkpoint A

Logic Overview (How)

Follows the same modular design as checkpoint b. Each file is responsible for a specific function such as user input, display control or hardware interaction. These fields are built around a certain main loop that connects all the files and syncs between user inputs and system state and hardware output. The software uses pigpio library for the i2c display and the time and math liberties for timing and signal processing. The main core files such as main menu_definitions navigation and ui_renderer manage the menu driven ui and system logic while hardware modules like fg_hardware and meter hardware and digipot control waveform generation and measurement.

Compared to checkpoint B this version introduces additional functions including sine wave generation using spwm techniques, frequency measurement using GPIO edge detection, and expanded menu options to support these features. There are a few experimental files such as spwm.py and sine_hardware.py and related scripts were developed to test different approaches for generating a sine wave overall the software maintains the same structured design as checkpoint b while expanding incorporating more advanced signal processing features.

The software operates through a continuous loop on the main program that reads input from the rotary encoder, updates the system state and controls hardware outputs in real time. At boot up the system initializes all hardware components, including gpio pins, SPI Connections, dac outputs, lcd display, and frequency measurement inputs. The user interacts with the system using the rotary encoder where scrolling and button presses are used to navigate menus and adjust values such as frequency amplitude and voaltage.

The menu system allows the user to access multiplied settings such as function generators. Ohmmeter, voltmeter, DC ref, and frequency counter measurement mode. When adjusting values the input_handler file enables sensitive scrolling with both fine and fast adjustments. The display is updated using ui_render and lcd_display ensuring clear and safe output(no flicker and artifacting).

Hardware control is handled by certain modules, the Function generator produces square waves using pwn and supports sine wave generation through spwm-based modules such as spwm file and sine_hardware. Measurement functions

are handled by the meter_hardware, which performs resistance measurements, sign comparator sweep, voltage measurements using a SAR ADC method, and frequency measurement using edge detection on the GPIO pins. The system constantly syncs hardware outputs with the software state and safely ensures outputs are turned off when navigating away from active does. THis allows real-time interaction while supporting the additional experimental introduced in this checkpoint.

main.py

The whole project is built as a modular RPi system. Main.py is the file that ties everything together. It is made up of imprints from other files, hardware initialization, a shared state dictionary, hardware synchronization function, and one continuous loop. In the loop, it reads inputs from the rotary encoder, processing every press and scroll and updating the LCD, calling measurement functions, and turns hardware outputs on or off. THis file is the brains of the entire project because every file either sends info into it or receives commands from it. It uses pigipio time sys(used to exit programs safely) and threading(allow to run things in parallel, continuous freq measurement and continuous sine generation and background updates without blocking the UI) , plus all the project modules.

See Deliverables/Checkpoint_A/code/main.py

- Hardware: lines 267 - 362
- Scroll Input: lines 205 - 225
- Button Press Input: lines 227 - 253

Menu_definitions

This is the map of the UI. Made of two main parts; the MENUS dictionary and DEAFULT_STATE dictionary. The menus dictionary defines how all menus connect including the main menu, function generator ohmmeter voltmeter dc reference and the added frequency measurement mode. The default state dictionary stores the starting values for settings like waveform type, frequency amplitude voltage outpost states and measurement values. THis file does not control hardware directly. It gives the rest of the program a fixed structure to follow.

Menu

```
MENUS = {
   "main": [
      "OFF",
      "Mode Select",
   ],
   "mode_select": [
      "Function Generator",
      "Ohmmeter",
      "Voltmeter",
      "DC Reference",
      "Frequency Measurement",  # new - A- rubric requirement
      "Back",
      "Main",
   ],
```

- This is just a snippet but other tree portions follow a similar structure

Default States

```
DEFAULT_STATE = {
   "current_menu": "main",
   "menu_selection": 0,
   "needs_update": True,

   # scroll value mode
   "value_scroll_mode": False,
   "value_scroll_label": "",
   "value_scroll_target": "",
   "value_scroll_step": 0.625,
   "value_scroll_min": -5.0,
   "value_scroll_max": 5.0,
   "value_scroll_live_hw": False,
   "value_scroll_original": 0.0,
```

- This is just a snippet but other recording variables follow a similar structure

Navigation

This controls movement through the menus. It is made of the menuNagavitor class, helper logic for snapping to allowed steps and functions like scroll select go_back and go_main. This file decides what happens when the user scrolls the wonder press button while not in values entry mode, it changes menus, turns outputs on and off, starts input editing and enforces rules like shutting outputs off when backing out of certain screens. This file works closely with menu definition, input_handler and main.

Back and Main

```
def go_back(self):
        # back navigation with rubric-required side effects
        cur = self.st["current_menu"]

def go_main(self):
        # main button - go all the way home and turn everything off
        # rubric: outputs must be off by default at main
        self.st["fg_output"] = False
        self.st["dc_output"] = False
        self.st["vm_source"] = "External"
        self.st["hw_update"] = True
        self.st["current_menu"] = "main"
        self.st["menu_selection"]  = 0
        self.st["needs_update"] = True
        self.st["ohm_park_needed"] = True
        self.st["freq_measuring"] = False
        self.st["freq_reading"] = None
        print("went to main - outputs off")
```

Selection Results

```
if picked == "Mode Select":
            s["current_menu"] = "mode_select"
            s["menu_selection"] = 0
```

- This is just a snippet but other selections follow a similar structure

Input_handler

This file is the value-editing helper. It is made of functions for starting scroll mode, changing a value, confirming a value, and canceling a value. This is what the system uses when the user is editing things like frequency, amplitude, or DC voltage. It stores labels, target variables ,steps size, range, and original values in the shared state so the rest of the program knows what is being edited. It does not draw anything itself and does not control hardware directly unless the state says to do live updates.

See Deliverables/Checkpoint_A/code/input_handler.py

ui_render.py

This is a helper file that defines a class that operates as a backbone for the display of all the functions across the testbench, handling the logic for what “lcd_diplay.py” needs to show. Its main function “update” checks whether a redraw is needed for the current display based on current menu location or selected test bench function.

```
def update(self):
    if not self.st["needs_update"]:
        return

    if self.st.get("value_scroll_mode"):
        self._draw_scroll_value()
        self.st["needs_update"] = False
        return

    cur   = self.st["current_menu"]
    items = MENUS.get(cur, ["Error"])
    sel   = self.st["menu_selection"]

    if not items:
        self.disp.write_menu("ERROR", ["empty menu"], 0)
        self.st["needs_update"] = False
        return
```

A good example for how this works is the “_draw_ohmmeter()” function. When the ohmmeter is selected through the menu, this function reads the resistance states and decides what to display whether currently reading, actively searching or producing an error. From here, “lcd_display.py” handles the actual display based on the logic.

```
def _draw_ohmmeter(self, items, sel):
    r   = self.st["ohm_reading"]
    u   = self.st["ohm_uncertainty"]
    err = self.st.get("ohm_error", "")

    if err:
        # "! Open/No resistor" = 19
        self.disp.render_lines([
            "=== OHMMETER ===",
            "! " + err[:18],
            "Range:500-10k ohm",
            "> " + items[sel],
```

```
            ])
        elif r == 0.0:
            self.disp.render_lines([
                "=== OHMMETER ===",
                "Measuring...",
                "Range:500-10k ohm",
                "> " + items[sel],
            ])
```

lcd_display

This is a low level lcd wrapper class. It is made of display class and methods for clearing the screen, writing one line, rendering multiple lines, and displaying menus. It uses the CharLCD aspect of RPLCD.i2c library to communicate to the 20x4 lcd. One important thing this field does is reduce flicker by rewriting lines that changed. That makes the screen cleaner and smoother. This fuels us using ui_render and main.

LCD Flicker Display

```
def render_lines(self, lines, force=False):
        # update only the lines that changed, limits lcd flicker and i2c traffic
        padded = [f"{line:<20}"[:20] for line in lines]

        for row in range(4):
            line = padded[row] if row < len(padded) else " " * 20

            if force or self._last_lines[row] != line:
                self.lcd.cursor_pos = (row, 0)
                self.lcd.write_string(line)
                self._last_lines[row] = line
```

encoder

This is the rotary encoder driver. It is made of rotary encoder class, a gray-code transition table, callback setup, and methods for reading rotation and button state. It uses pigpio callbacks to catch every encoder edge and then covers the transition into cw or ccw steps. This file is the main input source for the whole UI because it tells main what the user is doing with the knob and button.

Knob Input Detection

```
# checks for rotary encoder inputs
    def read_rotation(self):
        # atomically grab and clear accumulated steps
        s = self._steps
        self._steps = 0
        return s
```

Button Input Detection

```
# checks for rotary encoder button press
    def is_button_pressed(self):
        return self.pi.read(self.pbtn) == 0
```

Fg_hardware

This file controls the function generator hardware. It is made of pwn helper functions, amplitude to step conversion, SPI open/close functions and start/stop functions for the function generator. For square waves it uses sysfs PWN on GPIO 18. For sinewave it routes to sine generation code through spwm, it also uses SPI to change amplitude using the digiplot. This file works directly with main and indirectly with fg-sine and spwm.

Open/Close SPI

```
# opens SPI for function generator
def fg_open_spi(pi):
    return pi.spi_open(CE_FG_AMP, SPI_BAUD_FG, 0)

# closes SPI for function generator
def fg_close_spi(pi, sd_fg):
    sine_stop()
    _pwm_set(pi, 0)
    try:
        pi.spi_close(sd_fg)
    except Exception:
        pass
```

Start/Stop FG

```
# starts function generator - routes to sine or square path based on waveform
def fg_start(pi, sd_fg, frequency, amplitude, waveform=WAVEFORM_SQUARE):
    if waveform == WAVEFORM_SINE:
        # stop square before starting sine
```

```
        _pwm_set(pi, 0)
        sine_start(pi, sd_fg, frequency, amplitude)
    else:
        # stop sine before starting square
        sine_stop()
        freq = int(max(100, min(10_000, frequency)))
        _pwm_set(pi, freq)

        if sd_fg is not None:
            step = _amp_to_step(amplitude)
            # square amplitude goes to wiper 1 (0x10) of the dual digipot
            pi.spi_write(sd_fg, [DIGIPOT_SQUARE_WIPER, step])

        print(f"fg_start: {freq}Hz  amp_step={_amp_to_step(amplitude)}")

# stops function generator - stops both square and sine paths
def fg_stop(pi, sd_fg):
    sine_stop()
    _pwm_set(pi, 0)
    if sd_fg is not None:
        # zero both wipers on stop
        pi.spi_write(sd_fg, [DIGIPOT_SINE_WIPER, 0])
        pi.spi_write(sd_fg, [DIGIPOT_SQUARE_WIPER, 0])
```

Fg_sine

This file is a simple wrapper file for sine wave generation. It is made of two small functions, one to start sine output and one to stop it. These functions call sine_start and sine_stop from spwm. This file exists mainly to keep sine-specific code separated and make the man program cleaner.

Start Sine

```
def fg_sine_start(pi, sd_fg, frequency, amplitude):
    sine_start(pi, sd_fg, frequency, amplitude)
    print(f"fg_sine_start: {frequency}Hz  {amplitude}Vpp")
```

Stop Sine

```
def fg_sine_stop(pi, sd_fg):
    sine_stop()
    print("fg_sine_stop")
```

SPWM

This file is the key field for the sine wave. It is made of sine-wave pulse-building logic, pigpio waveform generation, frequency snapping, amplitude snapping, and start/stop functions. Instead of generating a sine wave directly, it builds a PWM-style waveform whose duty cycle changes in sinusoidal-like pattern. That waveform is then filtered by hardware to appreciate a sine-wave. This file works with fg_hardware and fg_sine files to provide sine output for the function generator, using math and pigpio.

Pulses

```
def _build_pulses(pi, frequency):
    dynamic_samples = max(10, 1_000_000 // (frequency * 10))
    total_period_us = 1_000_000.0 / frequency

    pulses = []
    current_time_us = 0.0
    for i in range(dynamic_samples):
        next_time_us = (i + 1) * (total_period_us / dynamic_samples)
        slice_duration = int(round(next_time_us)) - int(round(current_time_us))
        angle = (i / dynamic_samples) * 2 * math.pi
        duty_cycle = ((math.sin(angle) + 1.0) / 2.0)
        on_time = int(round(slice_duration * duty_cycle))
        off_time = slice_duration - on_time

        if on_time > 0:
            pulses.append(pigpio.pulse(1 << SINE_PIN, 0, on_time))
        if off_time > 0:
            pulses.append(pigpio.pulse(0, 1 << SINE_PIN, off_time))

        current_time_us = next_time_us

    return pulses
```

Frequency Function

```
freq = int(max(1000, min(10000, round(frequency / 500) * 500)))
```

Amplitude Function

```
amp = max(0.625, min(10.0, round(amplitude / 0.625) * 0.625))
amp_fraction = amp / 10.0
```

Output

```
# build and send waveform
    pulses = _build_pulses(pi, freq)
```

```
    pi.wave_add_generic(pulses)
    wave_id = pi.wave_create()
    print(f"wave_create returned: {wave_id}")

#LCD output
print(f"sine_start: {freq}Hz  amp={amp}V  step={step}  samples={len(pulses)//2}")
```

Sine_hardware

This file is another sine-related file, however it takes a different approach. It is made of an R-2R DAC pin map, sine lookup table generations, scaling functions, GPIO dac writing functions, and the sinegenerator class. Instead of spwm on one pin, this file describes generating a sine wave by outputting values across multiple DAC bits. It also includes frequency measurement support in the same file, this experimental implementation file used to develop the sine wave feature.

See Deliverables/Checkpoint_A/code/sine_hardware.py

- Configuration: lines 28 - 43
- Lookup Table: lines 45 - 65
- DAC Writing: lines 68 - 72

Digipot

This file is the digiplot helper. It is made of the DigitalPotentiometer class, calibrated constants, calculated coefficients, conversions for step resistance and resistance to steps, and spi write functions. This file does not control the whole ohmmeter itself but provides the math and control needed to set the digiplot accurately. It is useful anywhere resistance must be converted into the step values the chip understands.

Characterizing Coefficients

```
# characterized coefficients from bench measurements
    SLOPE = 72.79
    INTERCEPT = 104.28
    SERIES_R = 422
    MAX_STEP = 127
    MIN_STEP = 0
```

Step to Resistance

```
# step to resistance value conversion
    def step_to_res(self, step):
        if step > self.MAX_STEP:# upper bound step
            step = self.MAX_STEP
        if step < self.MIN_STEP:# lower bound step
            step = self.MIN_STEP

        res = (int(step) * self.SLOPE) + self.INTERCEPT
        return int(round(res))
```

Resistance to Step

```
# conversion from resistance to step value
    def res_to_step(self, res=10000):
        max_res = self.SLOPE * self.MAX_STEP + self.INTERCEPT
        min_res = self.INTERCEPT

        if res > max_res: # upper bound resistance
            res = max_res
        if res < min_res: # lower bound resistance
            res = min_res

        step = round((res - self.INTERCEPT) / self.SLOPE)

        if step > self.MAX_STEP:
            step = self.MAX_STEP
        if step < self.MIN_STEP:
            step = self.MIN_STEP

        return step
```

SPI Writing

```
# initializes SPI connection
    def __init__(self, pi, spi_channel=0, spi_speed=100000, spi_flags=0):
        self.pi = pi
        self.spi_activate = pi.spi_open(spi_channel, spi_speed, spi_flags)
```

DC_Reference

This file is a standalone DC reference program. It is made of DC Reference class, DAC code conversion, SAR-based voltage measurement, encoder reading, button handling, lcd updating, and its own main loop. The main loop is structured to run on its own and also be implemented into the overall main file without any conflicts. The

hardware connects to the user via the LCD and all other files were integrated as we progressed. It resembles an earlier isolated implementation of the dc reference mod before everything was folded into the larger menu driven system.

See Deliverables/Checkpoint_A/code/DC_Ref.py

- DAC Code → Voltage Conversion: lines 32 - 34
- Voltage → DAC Code Conversion: lines 70 - 73
- Code to DAC Handling: lines 64 - 68
- SAR Readings: lines 87 - 101
- LCD Connection: lines 120 - 146

SPWM_Signal, SPWM_sineWave, and sinsmaple

These are our test files or experimental files for sinewave generation. They are made of pulse generation math pigipio waveform creation timing calculations and simple loops to update or test sine-like outputs. These files show that different methods were being tried for sine generation before settling on the main integrated approach. They are useful for development, debugging and comparing waveform strategies, but they are not the main UI and controller files.

See Deliverables/Checkpoint_A/code/SPW_Signal.py
See Deliverables/Checkpoint_A/code/SPW_sineWave.py
See Deliverables/Checkpoint_A/code/sinsample.py

Full Functionality

Encoder reads user's knobs and button actions, main receives that input and updates shared states. Navigation or input handler changes menus or values ui_render decides what should be displayed, lcd_discplay writes it to the lcd and fg_hardware, meter_hardware, digiplot and the down files are made and how they work together

Full User Interface (UI)

Task Breakdown

- ☐ Create a navigation process for the User using the rotary encoder to interact with the UI
- ☐ Allow access to all hardware
- ☐ Follow the given UI tree structure from the project desc
- ☐ Add “Back” and “Main” return options after hardware selection
- ☐ Add individual input handling within selected hardware

Design Process

To design the User Interface, the first step was to reference the given tree structure within the deliverables. This broke down the individual steps required to traverse from one end of the tree to the other, with all median steps listed. Implementation of each step was done in a top down fashion. The tree was extended from the starting point all the way to the leaf points within the structure. Upon startup of the Raspberry Pi 4, the UI tree was shown with the original selections of “Mode Select” or “OFF”. After this was implemented correctly, the expansion began. If “Mode Select” was chosen, the user was taken to a new screen where they could scroll through all of the different hardware we had available. This included the Function Generator, Ohmmeter, Voltmeter, DC Reference, and Frequency Measurement. The menu also included the options to go back to the original menu or the Main menu which is the current stage. Each individual hardware can be selected via the rotary encoder push button, or navigated to via scrolling on the encoder. Every stage shown includes the “Back” and “Main” options, serving the same purpose throughout the tree. The function generator has the largest internal tree which is structured as follows. The type option allows the user to choose from a square wave, sine wave, or going back. The Frequency selection allows the user to input the desired frequency for the wave by scrolling to a value. The amplitude is the same as frequency but controls the amplitude of the output wave. The final section of the Function Generator is the output option where the user can select to turn the output on or off. The Ohmmeter starts reading once selected so the only options in this tree are the “Back” and “Main” selections. Voltmeter includes external, read from an input, and internal, read from the DC reference, voltage readings. The external begins the readings and the internal selection sends the user to the DC reference stage. This stage includes the ability to select the voltage value input - similar to the amplitude and frequency selection process of the function generator - and the output selection which is structured the same as the function generator output stage. The Frequency

Measurement option behaves similar to the Ohmmeter stage. Once the option is selected, the hardware starts collecting data and doesn't stop until either "Back" or "Main" is selected. If "Main" is selected at all, the outputs are turned off by default.

Tests & Results

Testing included all selection, scrolling, lcd updates, and hardware syncs with the User Interface. The tests concluded that all connections were stable and that the UI was effectively communicating with all parts of our physical circuit. Overall just testing if there were bugs going back and forth and making sure that there are no misspellings and the words were matched from the UI tree given to us.

Design Updates

As the UI got bigger, some things needed to be restructured and some files began to step away from pigpio as it began to lag due to memory usage and cpu usage. All files needed to be killed before running or the lcd would artifact, disrupting the User Interface interactions. Due to the pixel limitation for the lcd display we had to shorten a lot of words because they were longer than 20 characters.

PCB

Set-Up

Task Breakdown (What)

When designing a PCB in general, it's important to identify 3 components before beginning the design: design software, footprint library, and manufacturer.

- Design Software

The top PCB design software is KiCad, Autodesk EAGLE, and Altium Designer. All of these software are well known and are equally functional when designing the PCB, and as a UGA student, you have access to all of them for free under a student license. While KiCad is the best for entry-level designers, we used Altium for its in-built footprint library, rule uploads, and industry standardization, allowing both ease of use and easily transferable skills into a professional field.

- Footprint Library

While Altium has a comprehensive library of common parts, odds are that not all of the components will have simple footprints. Even though it is possible to create these footprints from scratch, several footprint libraries exist for many parts. We recommend using SnapEDA for all of your needed footprints as they do not only have near every part’s footprint symbol.

- Manufacturer

It's important to determine what PCB manufacturer you will be using before beginning the design of your board, since the minimum and recommended manufacturer requirements will dictate the limitations of your design. We recommend using JLC PCB since they offer cheap, fast, high-quality PCB manufacturing while being very communicative throughout the whole process. Furthermore, an open source rule file from their manufacturing is available for simple upload for the manufacturer's rule.

https://github.com/ayberkozgur/jlcpcb-design-rules-stackups

Design Process (How)

To get started, I recommend watching the following YouTube videos:

Basic tutorial https://www.youtube.com/watch?v=YTGzncKU5RY Common Mistakes https://www.youtube.com/watch?v=D0X76Kbf8fQ Hierarchical Design https://www.youtube.com/watch?v=RGlRsl8vy6k

This gives a beginner an overview of how to properly set up your PCB schematic and board file while ensuring proper board health. Furthermore, the final video introduces the concept of hierarchical board design, which drastically increases board cleanliness. Segmenting different circuits across each schematic, it allows for a clean overall board view with global variables as the design becomes more complicated.

Common Mistake

Common mistakes with board design often come from tracing. Outside of the recommended YouTube video material, it is important to keep these trace practices in mind. As a rule of thumb, traces should be short and efficient while eliminating sharp turns to reduce noise from antennas (see Fig. P1). Furthermore, the type of trace dictates these rules further.

- Signal Traces
  These traces transfer data through components and between ICs. Usually requiring a high response time, it's important to keep these traces short and thin to lower current draw and increase signal speed. For these traces, there should be NO trace angles below 90 degrees, since they act as antennas, drastically increasing noise through the board.

- Power Traces
  These traces supply power to components and ICs throughout the board, thicker than signal traces, in order to supply more current. When not using dedicated power planes, it's important to use a “fat bus”. This is one extremely large parent trace that acts as a power rail for the power supply, with thinner branches stemming off to supply power to different components.

Design Methodology (Why)

PCB design creates a permanent and professional device for your project and is the final design stage for this project. Ideally, this PCB should be updated alongside the developments of your breadboard to not only refine your skills as the project grows, but also to segment the final design into manageable pieces to design the PCB for your first project.

Note that for the following PCB design, we decided to challenge ourselves by constraining the space to mimic the supplied PiHat Protector. The PCB is not required to be limited in size, and greater space should be embraced to ensure signal integrity, project function, and easier manufacturing. All PCB designs come out different depending on how the components are traced, so the following comments will be a discussion about our journey and methods through PCB design for your reference when tracing the components yourself.

Checkpoint C/B

See References/PCB Photos/Checkpoint C-B

Task Breakdown (What)

The testbench project has several working circuits, so it is important to update the PCB alongside the breadboard design in order to avoid the complexity of implementing all of them on the PCB at the same time. The following images are PCB WIP schematics and boards as the final designs were fleshed out, alongside comments for working through the design.

Luckily enough, much of the heavy work has already been done when designing the circuit on the breadboard, since the circuit diagrams simply need to be replicated to the PCB schematic file. Much of what remains is finding the optimal way to route traces over the board for each component to function cleanly and fast.

Design Methodology

- Hierarchical Schematics

As outlined in the provided support material, hierarchical schematics allow for the abstraction of complex circuits to eliminate messy wiring when working with complex systems. By using this structure for the board, it allows for implementation of each section separate to the overall function of the board, allowing for simple and easy board implementation.

- Power Planes

PCBs are defined by their usage of layers, allowing the corsage of traces between the top and bottom signal layer, allowing signal traces to cross into sections

otherwise unavailable or providing access to power traces to pins otherwise accessible.

While the use of “fat bus” traces throughout the board can be used to provide power to several components, we found ourselves too constrained by our size challenge, and we were further worried about the high noise lines interfering with some of the high frequency traces. Therefore, we implemented 2 intermediate power planes for the new Virtual Ground and a segmented power plane between +/- 12V and 3.3V. On top of eliminating more high frequency noise alongside the signal layers, it allows for power access to be as simple as a via routed into the plane.

- Circuit Blocking

In order to reinforce hierarchical structure, each circuit was initially designed on the board in isolation to each other, optimizing for both space constraint and pin availability for inputs & outputs. From here, the “union” tool would be used to make the circuit move around as one whole block rather than individual pieces. Ultimately, this allowed us to work within the limited space restriction by moving around full circuits on the board and jigsawing them together with important outputs or common voltage supplies bunched together. It also improved the look and feel of the board, segmenting it into the important functions outlined by the project description.

- Different Vias

Vias are integral to PCB design in their versatility when accessing traces and planes across the layers of the board. Outside of those included within the component footprints, our board uses 3 main types: signal, trace, and assembly. Signal vias lie very close to manufacturer minimum requirements since they only serve as bridges for traces between layers, requiring minimum current pull and cleaning up traces throughout the board. Power vias are larger like their trace counterparts, focusing on proper current pull to power or ground components.

- Planning for the Future

Understanding the several amount of circuits remaining to implement into the design, we attempt to make our designs as compact as possible, leaving additional room for the rest of the functional components as we designed them.

Checkpoint B

See References/PCB Photos/Checkpoint B

Task Breakdown (What)

The following PCB contains all of the functions and circuits from checkpoint B. This includes a +/-12V power supply, multimeter circuits, an internal voltage

reference, and a square wave generator, all with the same resolution and variability as the breadboarded circuit.

Note that all of the connections are exact copies of the circuits designed over a breadboard, so many of the comments will discuss how to optimize your board design rather than designing. For circuit design, reference the previous “Checkpoint B” sections for better comments on the design process and methodology.

Design Methodology (Why)

We continued to follow the same methodologies outlined within the intermediate checkpoint discussions in “Checkpoint C/B”. Following these design principles allowed for seamless circuit integration as the circuit continued to grow in size and complexity.

Checkpoint A-

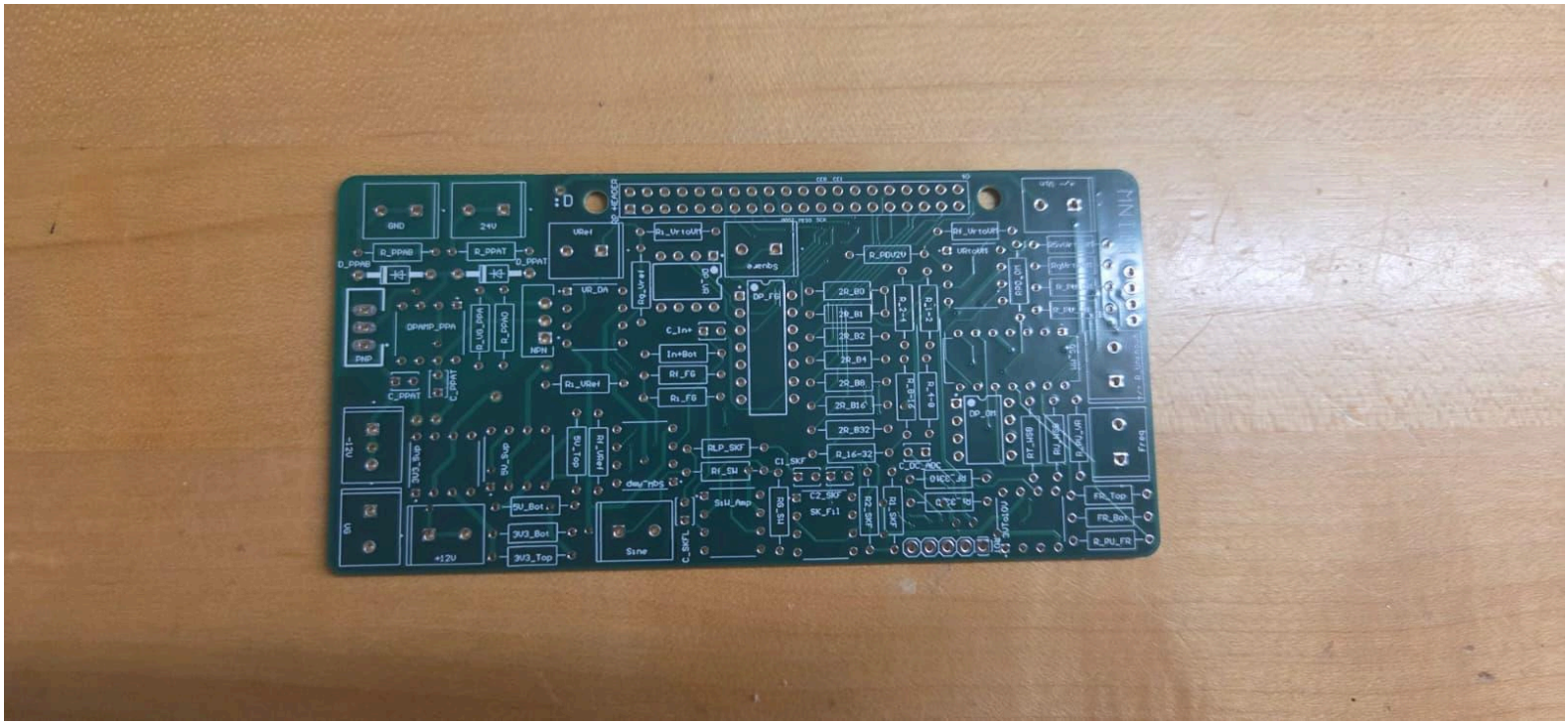
Task Breakdown (What)

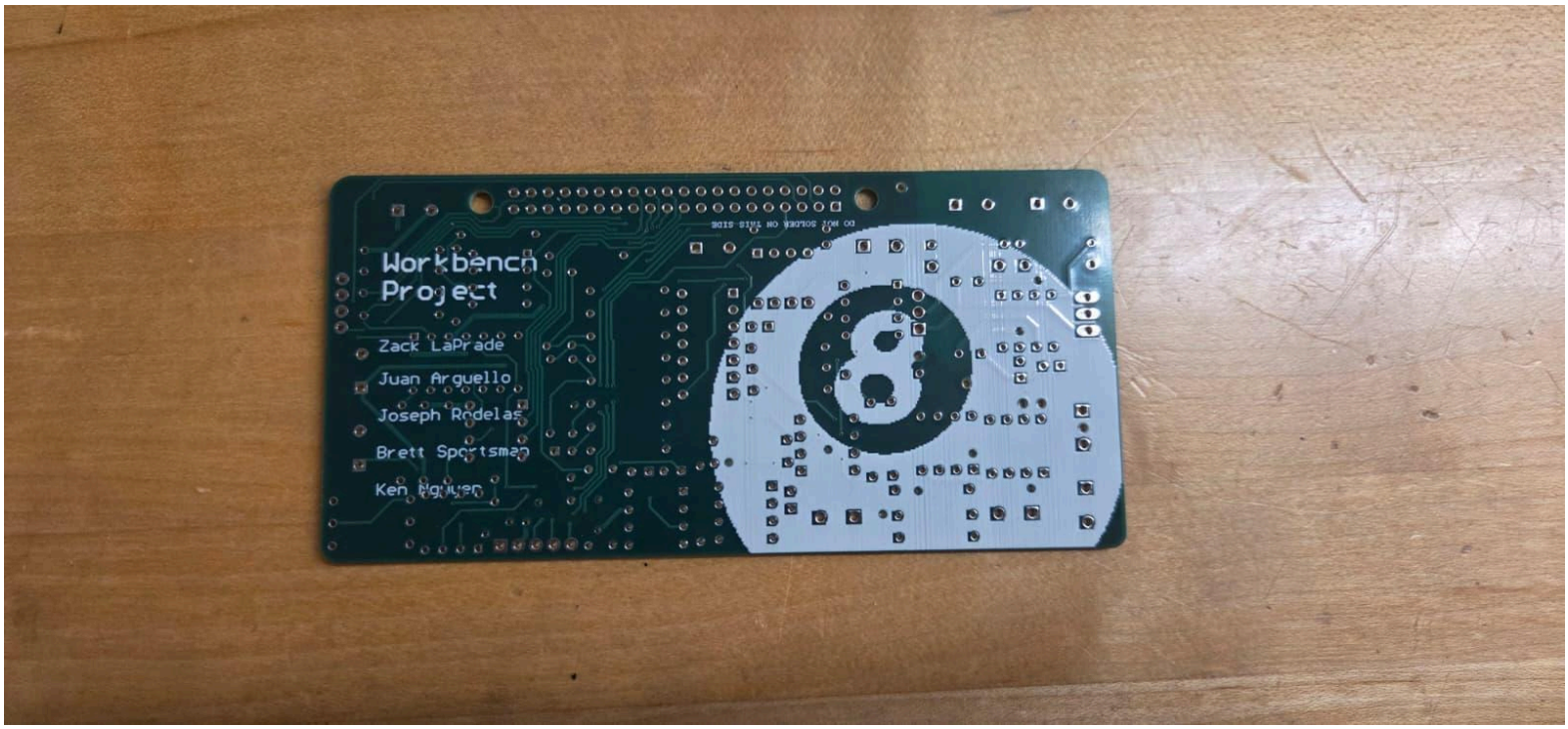
The following PCB contains all the functions and circuits from checkpoint A-, many of which were previously discussed in checkpoint B. New inclusions are 3V3 and 5V derived from the power supply instead of RPi4, a sine wave generator, and a frequency reader, all with the same resolution and variability as the breadboard.

Again, the following PCB contains all of the functions and circuits from checkpoint B. This includes a +/-12V power supply, multimeter circuits for the voltmeter and ohmmeter, an internal voltage reference, function generator circuits for square and sine wave, and frequency counter, all with the same resolution and variability.

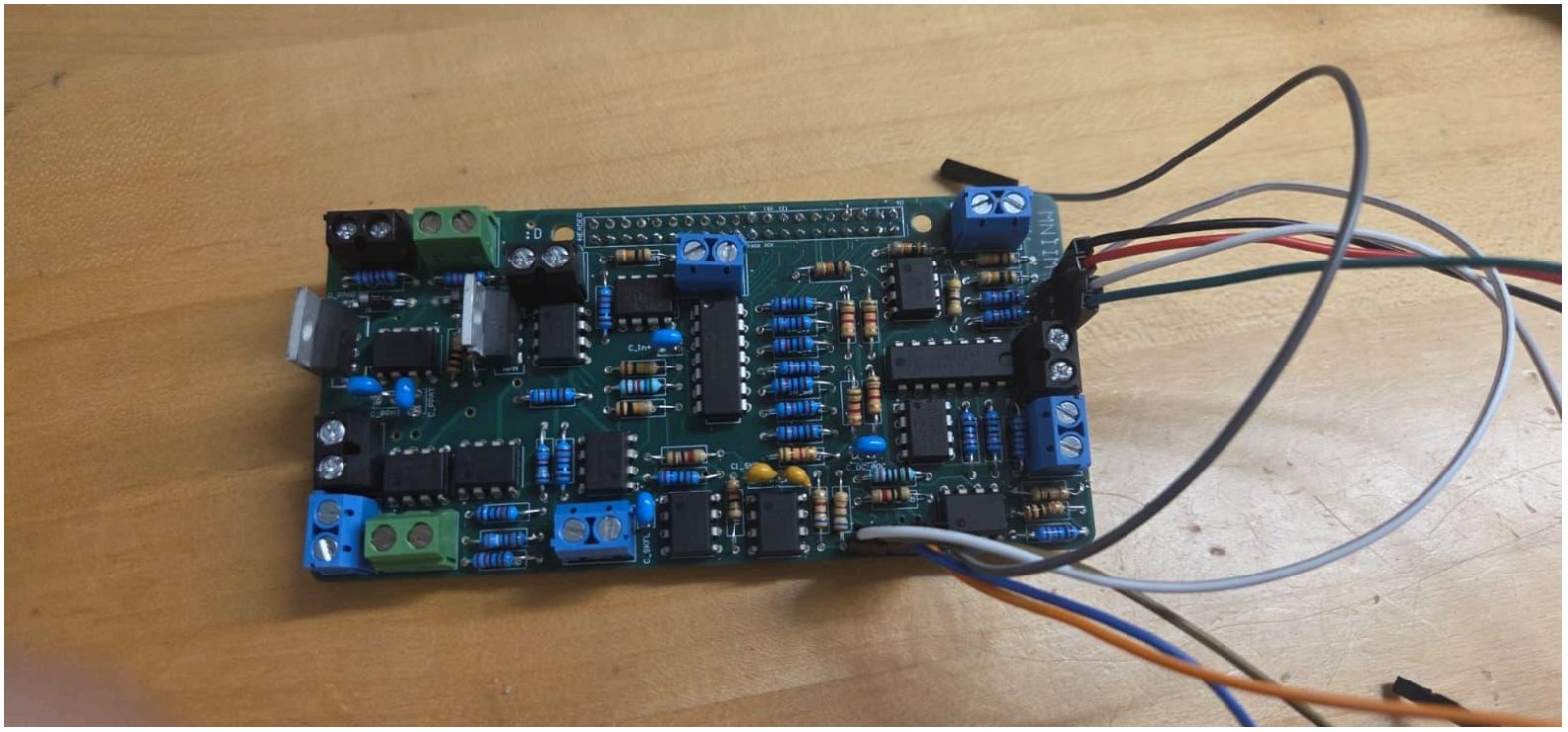
Note that all of the connections are exact copies of the circuits designed over a breadboard, so many of the comments will discuss how to optimize your board design rather than designing. For circuit design, reference the previous “Checkpoint A” and prior sections for better comments on the design process and methodology.

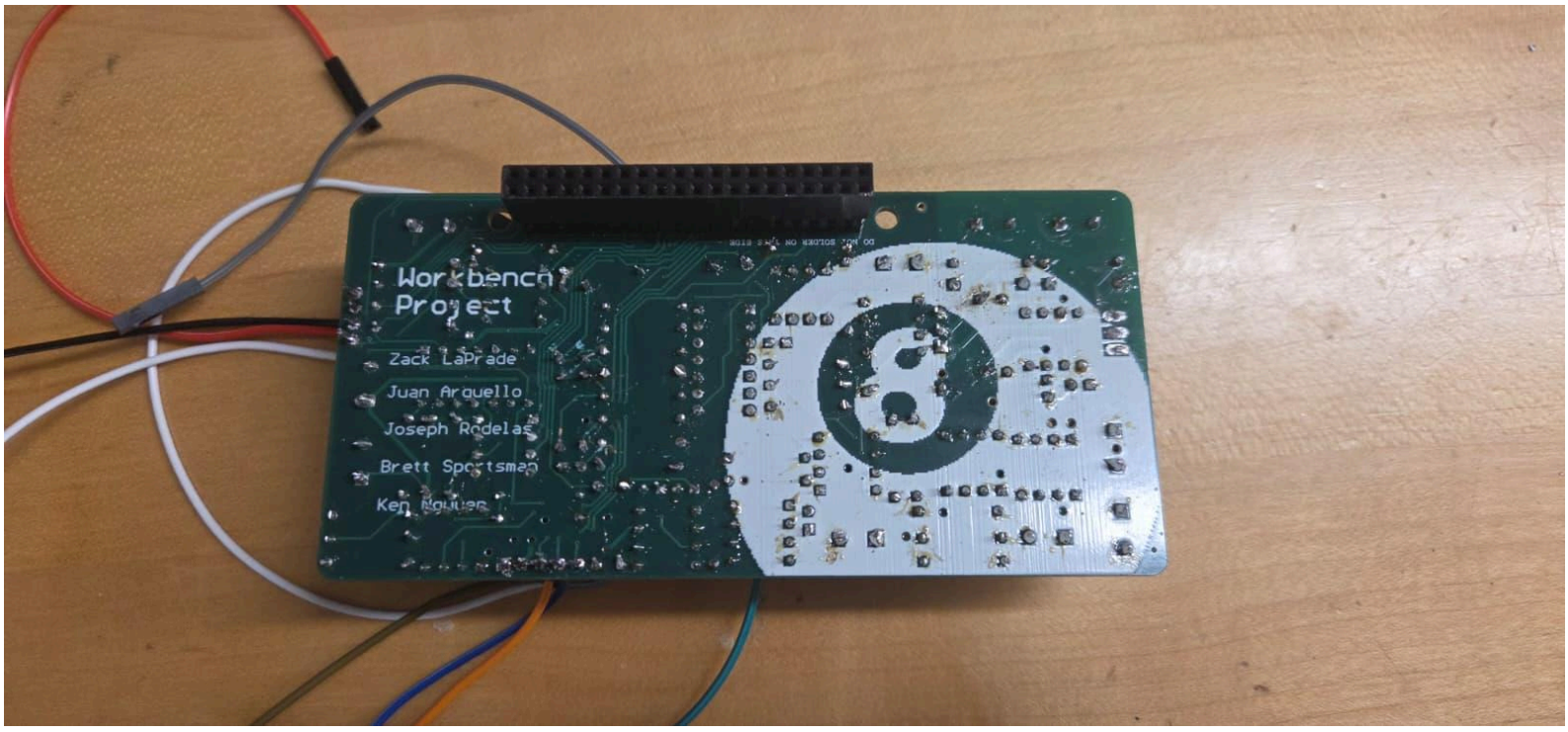
PCB Photos





#Manufactured PCB





#Fully Assembled Manufactured PCB

Tests & Results

Due to pressure to make a checkpoint presentation on time, the board assembly was hastily soldered. When testing the components, the power supply did not properly hold the 12V virtual ground and was subsequently not used for the checkpoint. The board will be resoldered in order to ensure proper connections and will be tested to ensure functionality.

Design Methodology (Why)

We continued to follow the same methodologies outlined within the intermediate checkpoint discussions in “Checkpoint C/B”. Following these design principles allowed for seamless circuit integration as the circuit continued to grow in size and complexity.

While many of the same design methodologies outlined in previous discussions were followed, like hierarchical schematics, power planes, and proper via usage. Some important changes were made to ensure the board functions properly and arrives on time.

- Electrical Isolation

One design flaw with the power supply design on the board was the power planes used to connect all the power pins, attempting to isolate high-frequency noise from the circuit. Over the breadboard, the push-pull amplifier, output & ground resistors, and decoupling capacitors had done much of this work for us ahead of time.

The real isolation needed to come from the pre-amplified sources and the rest of the board. Rather than having each of those power pins tap directly into the universal +/-12V (0/24V) and virtual ground (12V) would have actually increased the noise seen by the power before it went through the op-amp. To fix this, we electrically isolated the split rail power supply from the rest of the board, leaving only the post-split +/- 12V and virtual ground to feed into the power planes for these universal supplies.

- Assembly Pressure

Due to manufacturing and shipping time, the circuit designs for checkpoint A needed to be completed at least a week prior to the presentation day, and since our initial designs were lower quality that we had initially hoped, we gave ourselves less time to squeeze the Checkpoint A components onto the already full main board than we would have liked. As a result, rather than holding the union groups as a tried and true block when integrating the circuit, they served more as general zones that each component would sit in, with some components spilling into spaces as necessary. Furthermore to shorten assembly time from looking at the schematics with the previous silk mask labeling, we opted to instead place the component labels directly on or near the component footprints, referencing a master list to know which

components to solder where. While this reduces the look and feel of the device, a quick turnaround for assembly became our top priority, nearing the final presentation.

Ethics

Generative AI

Addressing the elephant in the room, generative AI like ChatGPT, Gemini, and Claude are an extremely sharp double-edged sword when it comes to research, design and troubleshooting. These resources are extremely responsive in terms of tailoring answers within users context, so good in fact as to where a couple prompts can develop the whole project with minimal effort on the designer's side.While initially very appetizing to prompt through the whole project, it is important to remember that AI doesn't truly understand the content it creating, and that the use of generative AI in this form factor is inherently unethical. For the purpose of a design class, using AI to find and generate design beats goes beyond the agreement for your team to design the device and defies the purpose of the class. Furthermore, it can result in longer troubleshooting periods as the design's generated aren't 100% correct.

However, this is not to say that AI isn't one of your greatest tools, as long as it's used within the bounds of research and troubleshooting, NOT full design. It must be used as an assistant to your critical thinking, not as a replacement. For the purposes of researching, AI can be used ethically to reveal popular circuit designs that follow desired function, but it is important not to let AI do all the research for you. Rather, AI should be used as a guide, showing you common paths of where to begin your design. When designing circuits post-research phase in simulation or over breadboard, Ai can be used again as a supportive tool to highlight common errors within the design, but it is important again to ensure it doesn't replace human reasoning. When prompting, it is important to not simply state the errors for the AI to find solutions for, but one should explain errors with their errors and current understanding of how to move forward, only using AI to find further areas to check and troubleshoot yourself. This is especially pertinent for software implementation. When writing code, AI should be used to show example code and common structures to implement, acting only as an aid as the programmer builds up the code function as a whole. Ultimately, the design must be accelerated with the use of generative AI, not created with minimal input from the design team.

Self Critiques

One of the largest critiques for our group would be how long it takes for us to get a method for design creation. Our first meetings were very inefficient due to Parkinson's law, where we set a large amount of time to complete the initial tasks such as setting up the digital potentiometers. Due to these earlier tasks being much easier then the later tasks past checkpoint C, we got very used to slacking off and taking large amounts of time to complete small tasks. This would eventually hurt us past checkpoint C until about Deliverable 8 whether we started dividing tasks more appropriately and setting more rigid amounts of time to complete the smaller tasks.

Along with that, a smaller critique would be how messy our breadboarded circuit was in the end. While we thought it was very impressive that we were able to get all of the deliverables to fit all onto two breadboards, it ended up making the later checkpoint harder to debug the circuit due to visibility. In the future, while we enjoyed the challenge of getting it onto 2 breadboards, it would be more beneficial and save time to spread it all onto 3 or more breadboards for easier debugging.

Rev. 2 Notes

If we were to restart again there would be several areas that we would improve. To begin, we should avoid using pigpiod. This library worked amazing all the way until we put the Raspberry Pi in standalone mode the night before checkpoint B, where it would get stuck. We never came to a formal conclusion on why we weren't able to use pigpiod in standalone mode as we ended up using another library, however we would avoid it due to it being unreliable.

Next, we would actually change the placement of the digipot from the Ohmmeter to the ADC. This discussion came up during our panel and had been something we had discussed even as we were implementing the design. While our thoughts were to minimize the noise of the board and get the closest measurement possible to the unknown resistor, the same solution could have been achieved with our current ADC and a voltage divider. With the unknown resistor in the voltage divider, the ADC would sweep through until the voltage value was higher than the voltage across the resistor. Then using digital signal processing, we could convert the voltage to resistance using a transfer function. The digipot would then be used for the ADC, eliminating wires that could potentially get disconnected on accident and having a consistent sweep.

Finally, as stated above, we would not try to cram the project onto two breadboards. While again we thought it was impressive that we were able to achieve this, it was completely unnecessary and made debugging much harder. Along with that, the components being so close would create more noise if instead the components were more appropriately spaced apart.

Appendix

Checkpoint C

Fig. C1 - Digipot pinout and commands

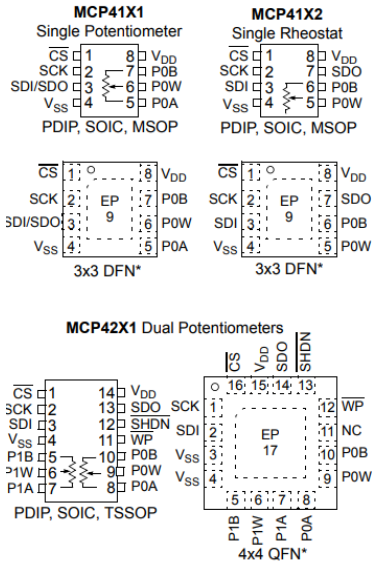
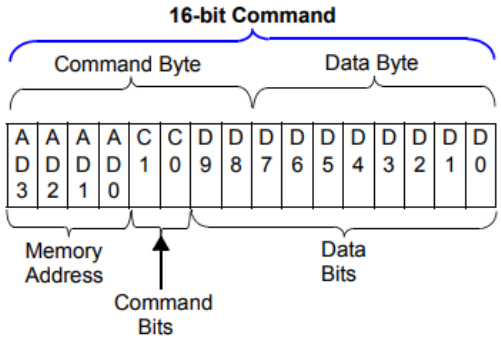
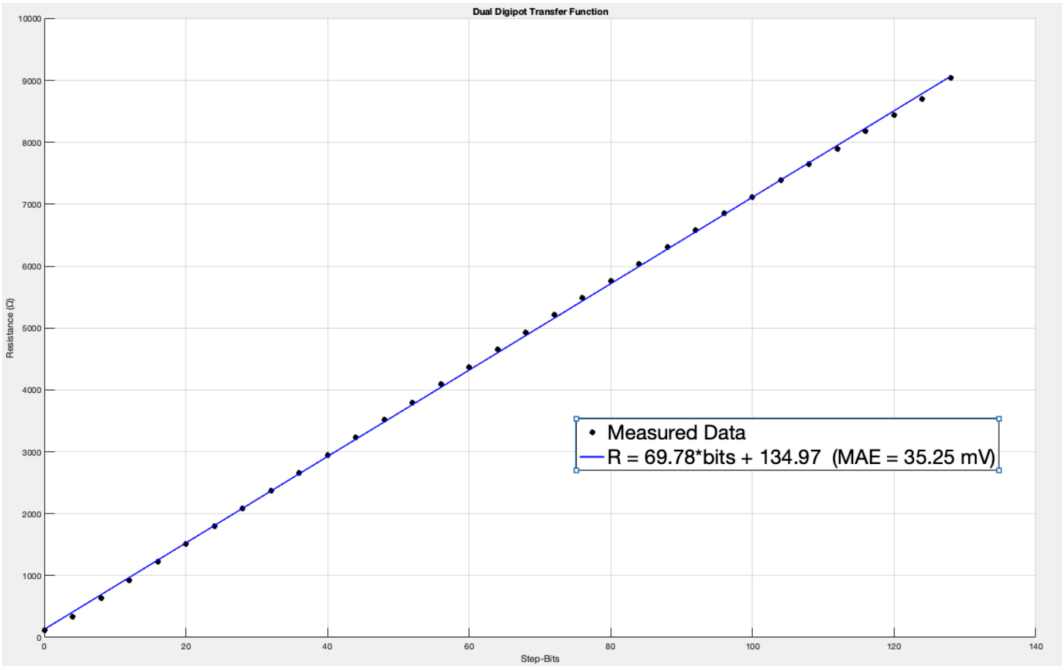
Pin Diagram	Device Commands
 #Single = MCP4131. Double = MCP4231	 **TABLE 7-1: COMMAND BIT OVERVIEW** C1:C0 Bit States / Command / # of Bits 11 — Read Data — 16-Bits 00 — Write Data — 16-Bits 01 — Increment — 8-Bits 10 — Decrement — 8-Bits

Fig. C2 - SPI processes

Functions	Brief Explanation
spi_open(spi_channel, baud, spi_flags) | | MISO | MOSI | SCLK | CE0 | CE1 | CE2 | | Main SPI | 9 | 10 | 11 | 8 | 7 | - | | Aux SPI | 19 | 20 | 21 | 18 | 17 | 16 |	Callable object to open the SPI pins on the RPi 4 for communication. spi_channel selected = 0 (main = CE0) baud = communication rate. - Selected: 10000 Spi_flags = defines spi mode - Selected : 0

spi_write(handle, data) `pi.spi_write(1, [2, 192, 128])`	Calls handle for the method of how to send a set amount of bytes through SPI handle: defined previously data: [(write command = 0x00), (data = 0-127 for full steps)]
spi_close(handle) `pi.spi_close(h)`	Placed at the end of the function to ensure SPI isn't over cluttered with information

Fig. C3 - Characterization of Digipots (MCP4231 and MCP4131-103)



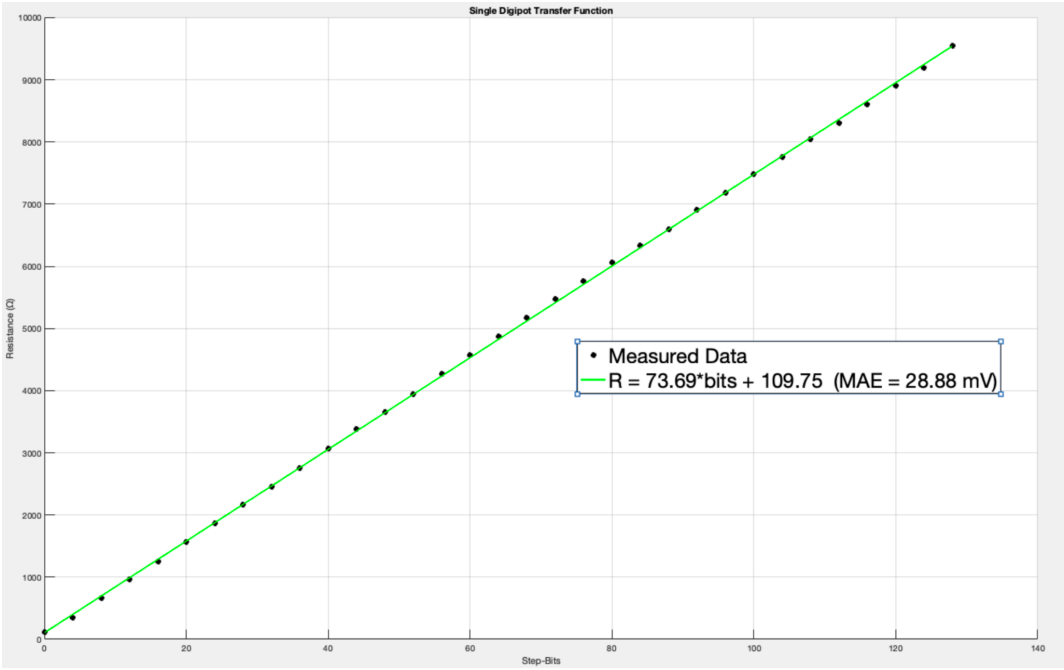


Fig. C4 - KY-040 pin functions

Pins	Function
CLK	Clock. Sends a square wave output signal alongside DT as the dial spins
DT	Data. Outputs at phase shift 90 deg. from CLK to indicate direction.
SW	Switch. Acts as the input for the button click.
+	Power Supply. Set to the same logic level as RPi4.
GND	Ground. Wire to the common ground of RPi 4.

Checkpoint B

Fig. B1 - Push pull amplifier

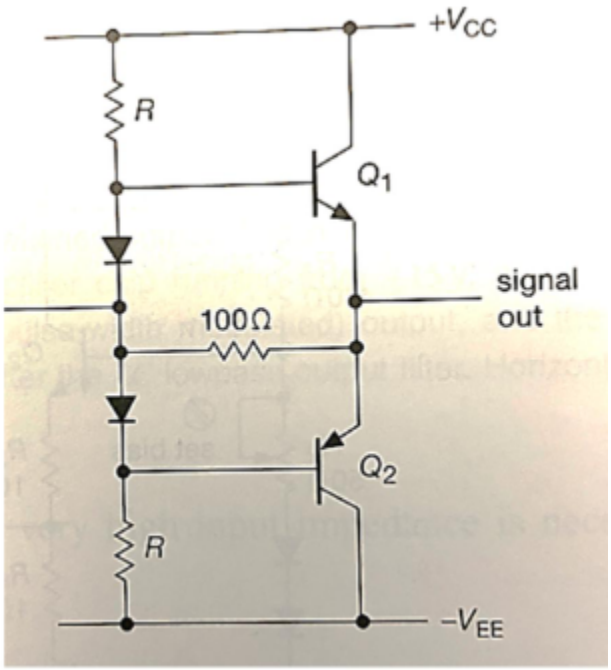


Fig. B2 - Power supply rails under 250Ω load

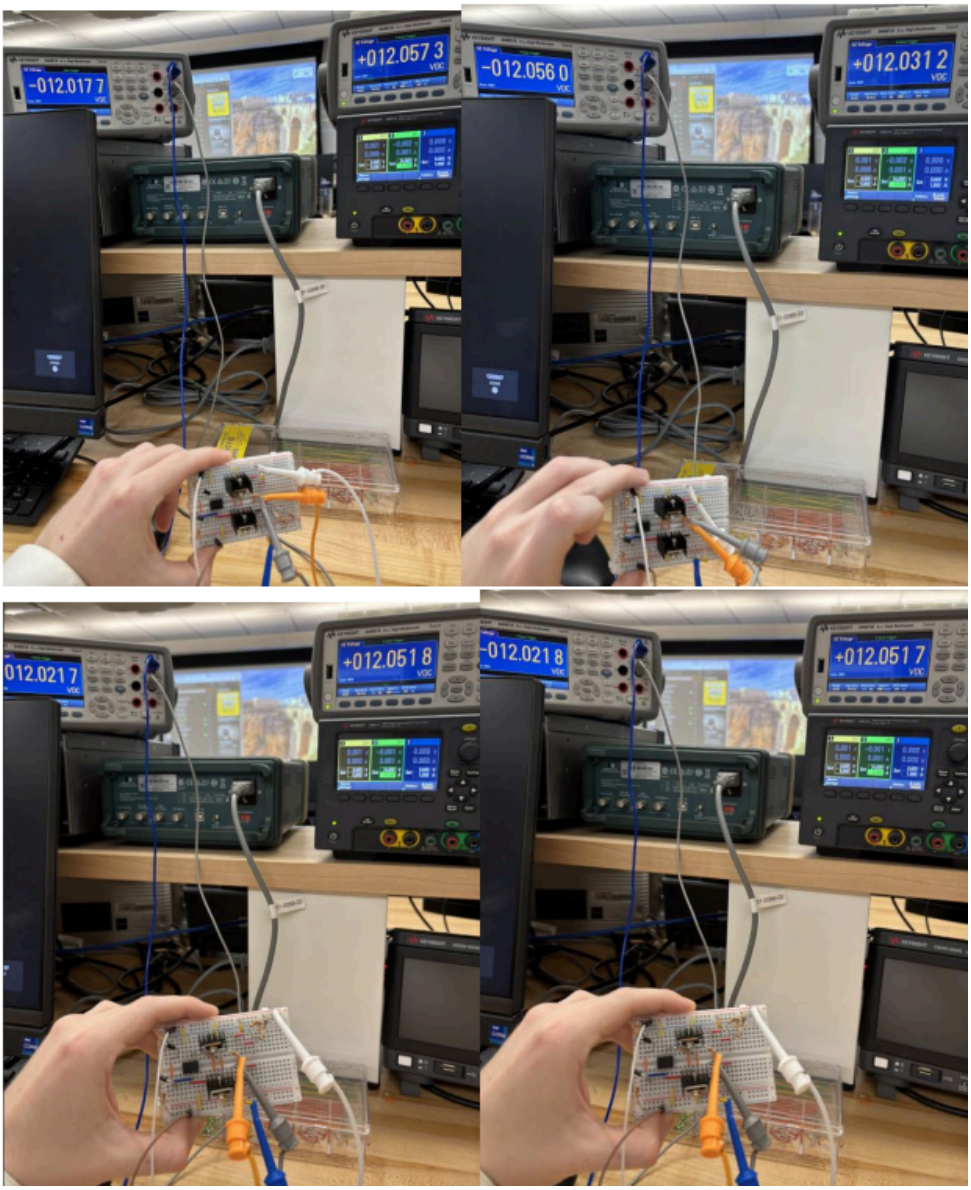
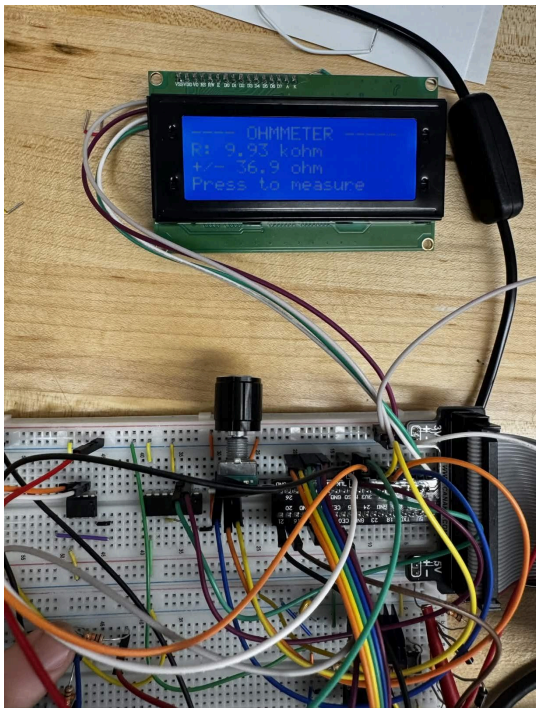
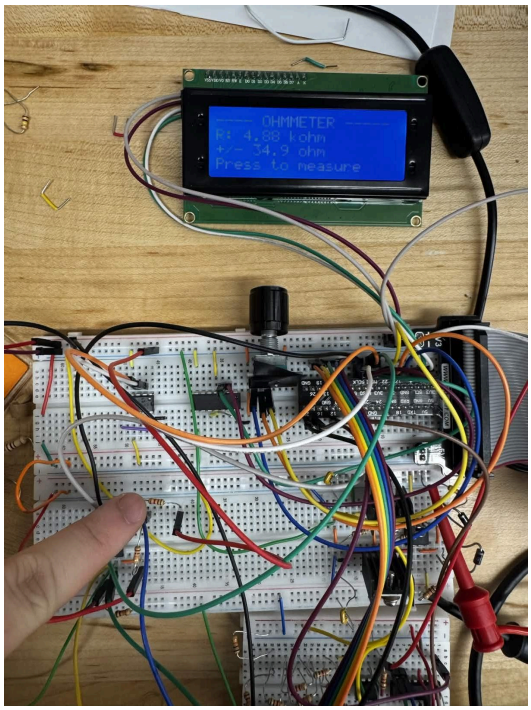


Fig. B3 - Ohmmeter working (10kΩ, 5kΩ, 550Ω)





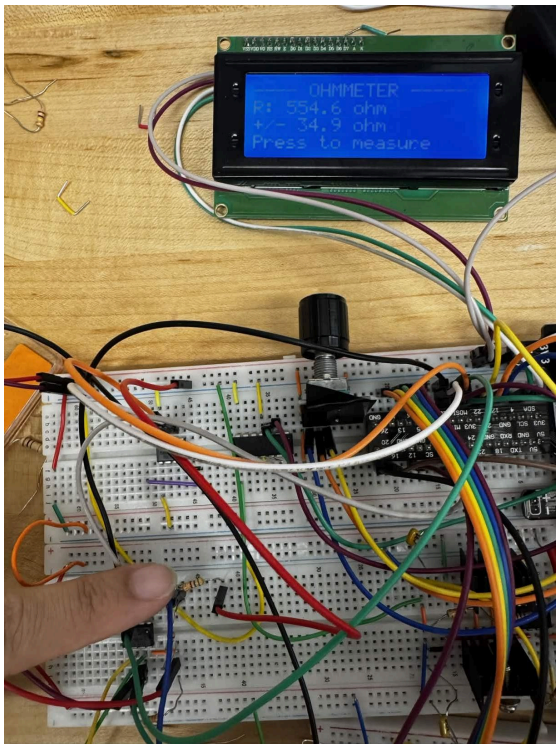


Fig. B4 - Summing Amplifier

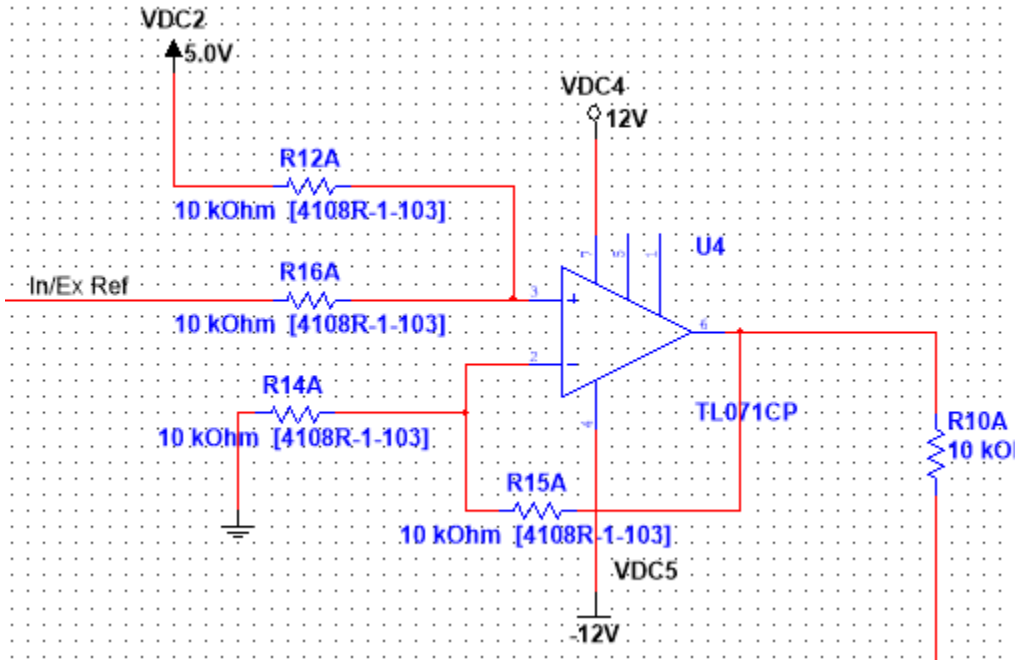


Fig. B5 - Voltmeter R-2R with Op-amp and comparator

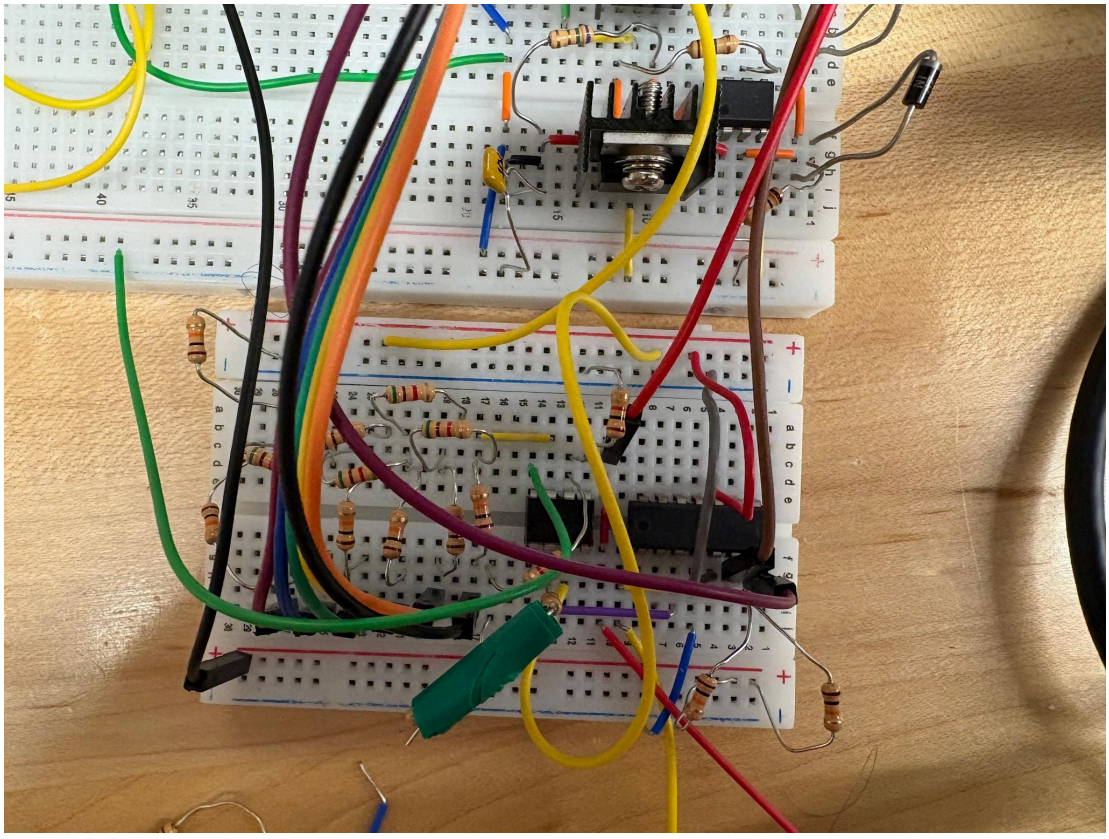
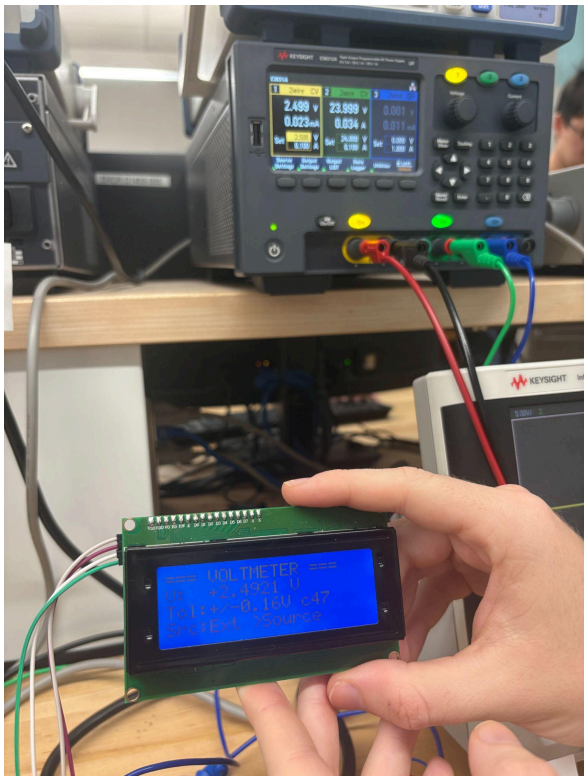


Fig. B6 - Voltmeter working (2.5V and -2.5V)



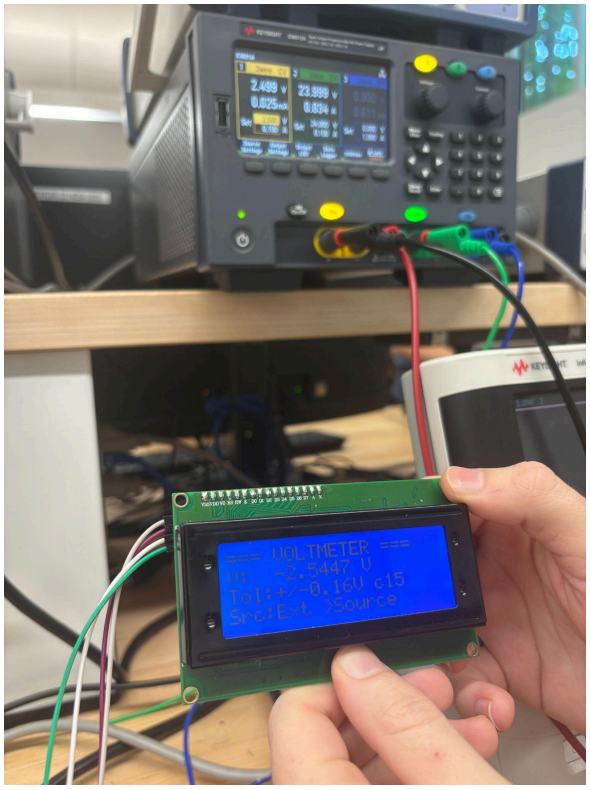


Fig. B7 - Square wave working (5.6kHz 10V peak-to-peak)

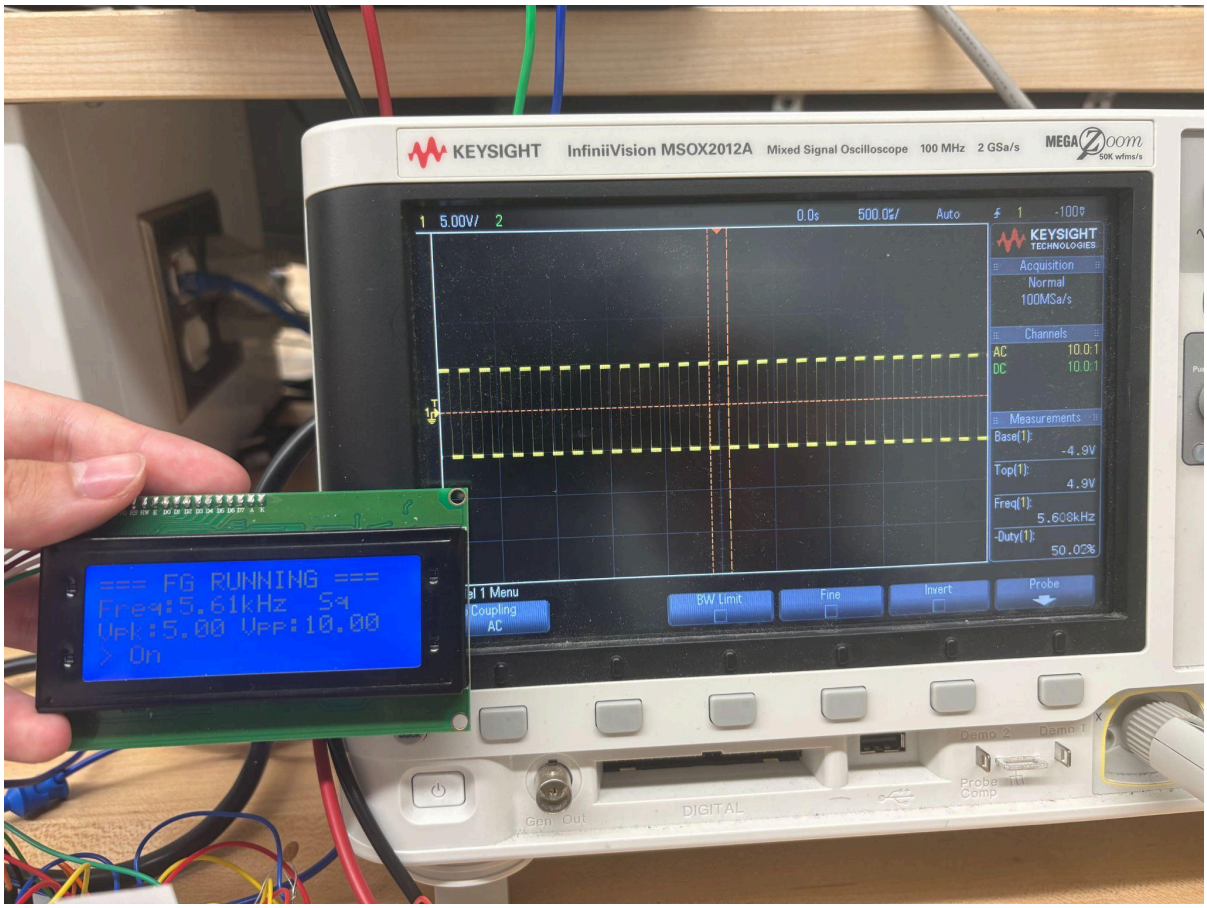
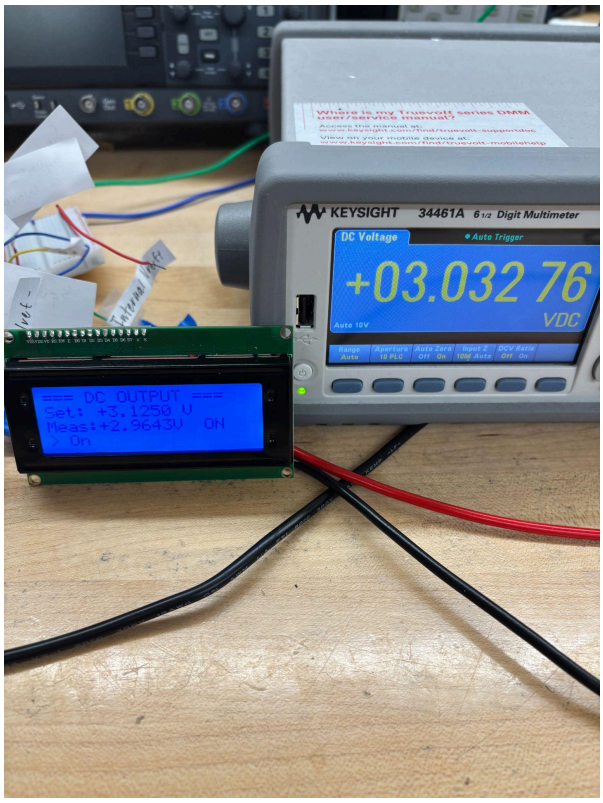
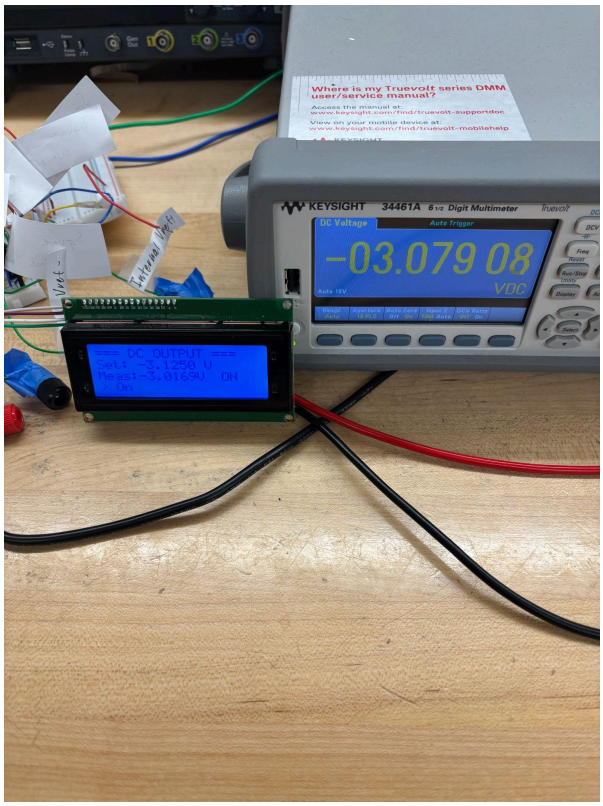


Fig. B8 - Voltage reference working (Output of +/-3V)





Checkpoint A

Fig. A1 - Sine wave working (10V 10kHz)

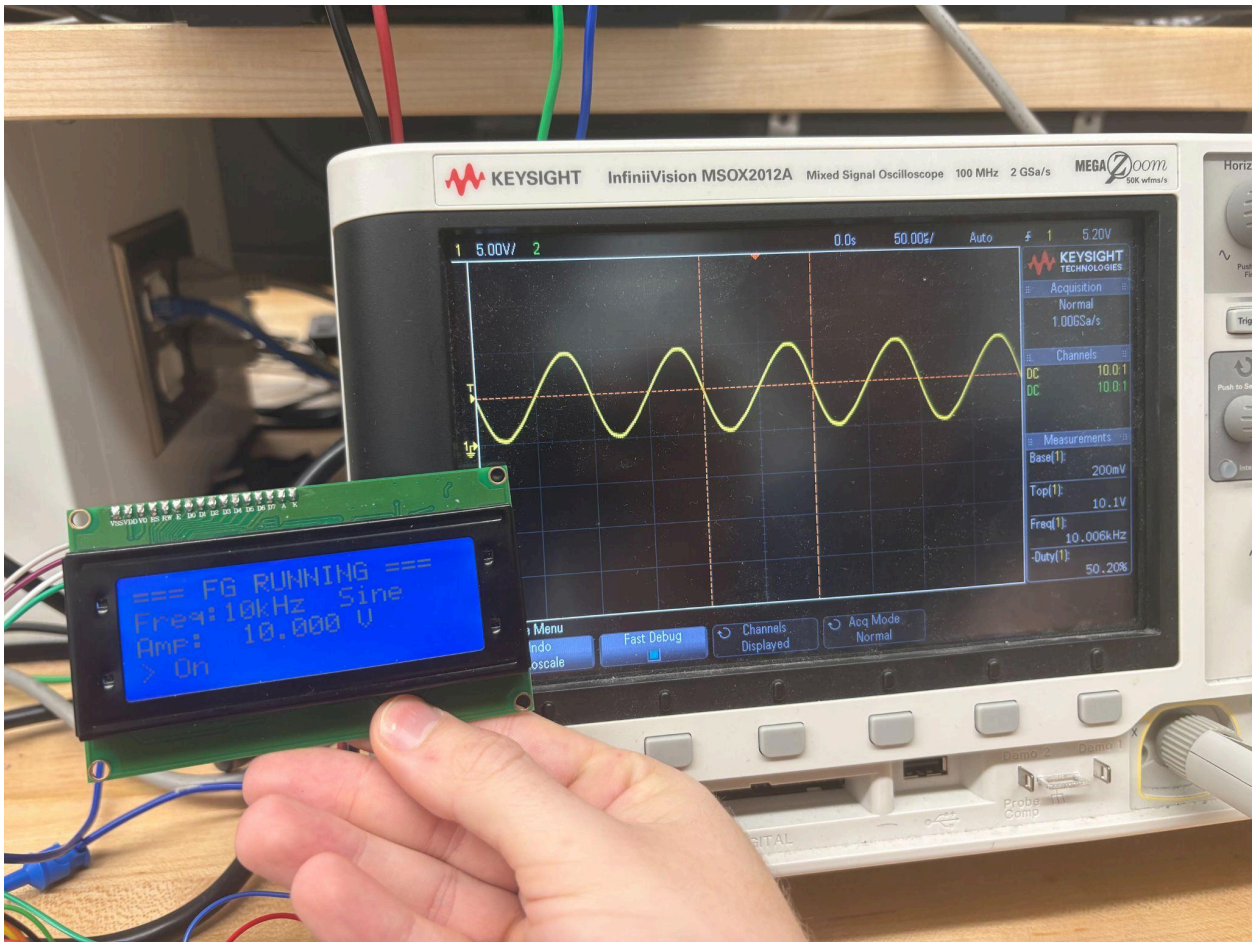
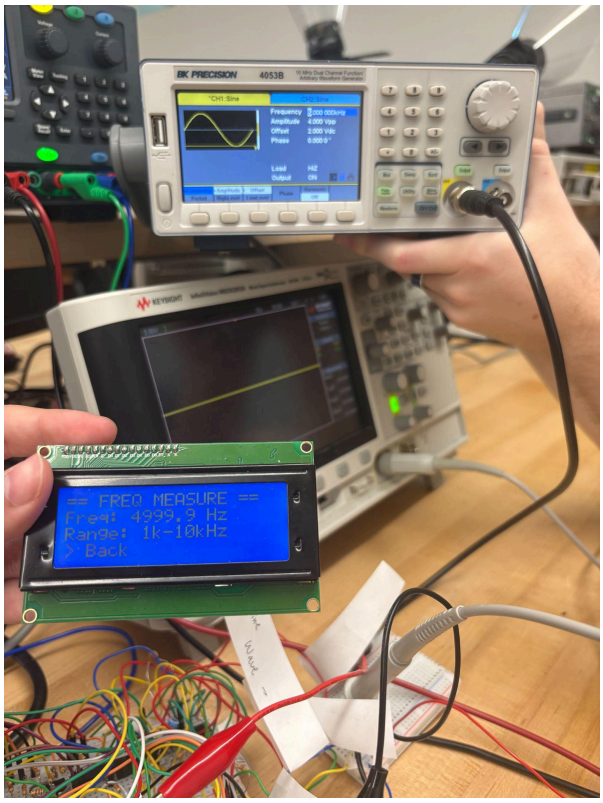
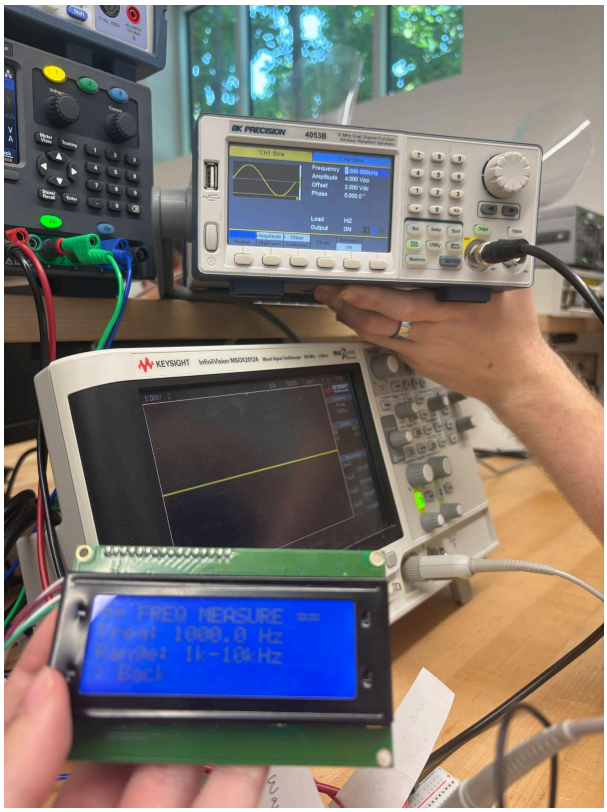


Fig. A2 - Frequency measurement (5kHz, 1kHz)





Software

Fig. S1- pigpio Python Library Documentation

http://abyz.me.uk/rpi/pigpio/python.html
Used for:
Connecting to the pigpio daemon: pigpio.pi()
Setting GPIO pin direction: pi.set_mode(pin, pigpio.INPUT)
Enabling internal pull-up resistors: pi.set_pull_up_down(pin, pigpio.PUD_UP)
Reading a pin state: pi.read(pin)
Main loop timing pattern using time.sleep()
Proper shutdown: pi.stop()
These calls appear in encoder.py and main.py.

Fig. S2 - RPLCD Library Documentation

https://rplcd.readthedocs.io
Used for:
Initializing the display: CharLCD('PCF8574', address, cols=20, rows=4)
Moving the write cursor: lcd.cursor_pos = (row, col)
Writing text: lcd.write_string()
Clearing the screen: lcd.clear()
All RPLCD calls are wrapped inside lcd_display.py so the rest of the program never calls the library directly.

Fig. S3 - PJRC Encoder Library - Quadrature Encoder Logic Reference

https://www.pjrc.com/teensy/td_libs_Encoder.html
Used for:
Understanding the 2-bit gray code state machine for quadrature encoders
The "combined" bit-packing trick:
combo = (prev_encoded << 2) | encoded
This puts the previous 2-bit state in the top bits and the current 2-bit state in the bottom bits, giving a 4-bit pattern that uniquely identifies every valid CW and CCW transition.
The specific CW pattern set: (0b1101, 0b0100, 0b0010, 0b1011)
The specific CCW pattern set: (0b1110, 0b0111, 0b0001, 0b1000)
Implemented in encoder.py read_rotation() method.

PCB

Fig. P1 - PCB Traces

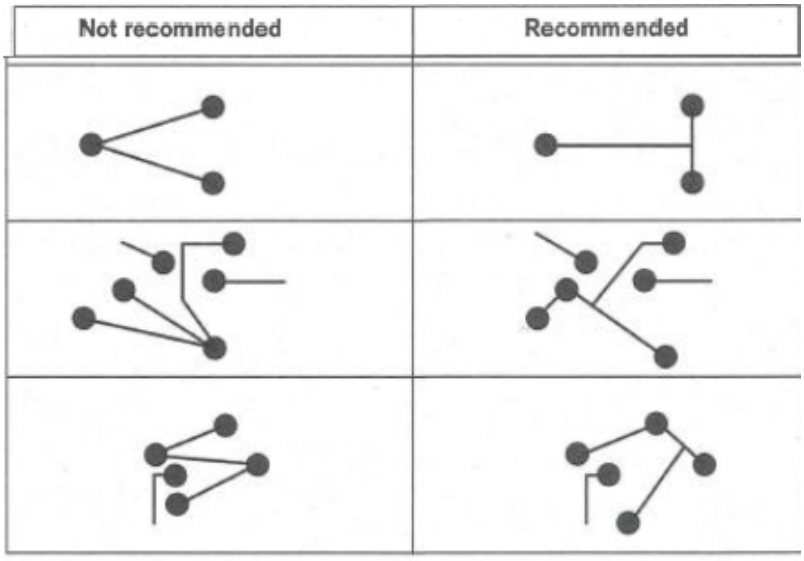


Fig. 5.10 Preferred routing pattern of conductors

